

Compact, Efficient and CCA-Secure Updatable Encryption from Isogenies

Antonin Leroux^{1,2} and Maxime Roméas³

¹ DGA-MI, Bruz, France

² IRMAR - UMR 6625, Université de Rennes, France

`antonin.leroux@polytechnique.org`

³ ANSSI, Paris, France

`maxime.romeas@ssi.gouv.fr`

Abstract. Updatable Encryption (UE) allows ciphertexts to be updated under new keys without decryption, enabling efficient key rotation. Constructing post-quantum UE with strong security guarantees is challenging: the only known CCA-secure scheme, COM-UE, uses bitwise encryption, resulting in large ciphertexts and high computational costs. We introduce DINE, a CCA-secure, isogeny-based post-quantum UE scheme that is both compact and efficient. Each encryption, decryption, or update requires only a few power-of-2 isogeny computations in dimension 2 to encrypt 28B messages, yielding 320B ciphertexts and 896B update tokens at NIST security level 1—significantly smaller than prior constructions. Our full C implementation demonstrates practical performances: updates in 28ms, encryptions in 48ms, and decryptions in 86ms. Our design builds on recent advances in isogeny-based cryptography, combining high-dimensional isogeny representations with the Deuring correspondence. We also introduce new algorithms for the Deuring correspondence which may be of independent interest. Moreover, the security of our scheme relies on new problems that might open interesting perspectives in isogeny-based cryptography.

Keywords: Updatable Encryption, Isogenies, Deuring Correspondence, Post-Quantum Cryptography

1 Introduction

Updatable Encryption (UE), introduced by Boneh *et al.* in 2013 [6], extends symmetric encryption by allowing ciphertexts to be re-encrypted under a new key without first decrypting them. Instead of downloading, decrypting, and re-encrypting large datasets, a client with the current key can generate a compact *update token* that enables an untrusted server to update all ciphertexts to the new key. This is especially valuable in cloud storage, where key rotation is essential to mitigate the impact of key compromise, but bandwidth and computational costs make naïve re-encryption impractical. In this work, we focus on the *ciphertext-independent* variant of UE, where a single token can update all ciphertexts under a given key. Beyond efficiency, UE must ensure *post-compromise security* and *forward secrecy* even if encryption keys and tokens may be exposed.

UE security definitions extend those for symmetric encryption to capture these stronger guarantees. Security is typically modeled over *epochs*, in which keys and update tokens may be adaptively revealed to the adversary. Early works proposed chosen plaintext attack (CPA) notions, such as IND-ENC and IND-UPD [30], which separately require indistinguishability of fresh encryptions and updated ciphertexts. Stronger notions, such as IND-UE [8], require indistinguishability between fresh and updated ciphertexts, thereby ensuring both confidentiality and unlinkability across key updates. In the classical setting, UE schemes secure under chosen ciphertext attacks (CCA) can be obtained by combining CPA-secure UE with ciphertext integrity (CTXT), using generic transforms such as Encrypt-and-MAC or Naor–Yung [28]. Achieving CCA security in the post-quantum setting, however, has proven significantly more challenging.

Lattice-based UE schemes have been built from the Learning with Errors (LWE) assumption, starting with Jiang’s *LWEUE* [25]. Nishimaki’s *RtR* [38] was the first ciphertext-independent UE scheme that is *unidirectional*, meaning that the new key cannot be recovered from the old key together with the update token. Nishimaki showed that unidirectional UE schemes have stronger security than plain UE. Unlike *RtR*, the UE schemes introduced in this work are not unidirectional. Lattice-based UE schemes use key-homomorphic public key encryption to enable ciphertext updates and, when also plaintext-homomorphic, can achieve unidirectional security. However, they face two critical limitations. First, they only support a bounded number of updates as ciphertext noise grows with each update. Second, using their homomorphic properties and knowledge of update tokens, an adversary can craft related-message ciphertexts, breaking CCA security. Consequently, all known lattice-based UE schemes achieve at most IND-UE-CPA security.

Cryptographic group actions, particularly isogeny-based ones, offer a compelling post-quantum alternative. Candidates such as CSIDH [9], SCALLOP-HD [12], and CLAPOTI [39] are believed to be quantum-resistant. Leroux and Roméas [34] adapted the classical UE scheme *SHINE* [8] to this setting in their *GAINE* construction, which could in principle achieve CCA security if instantiated with a group action satisfying both *mappability*—existence of an efficient bijection between the acted-upon set and bit strings—and *weak pseudorandomness* [2]. Unfortunately, no known post-quantum group action possesses both: isogeny-based actions lack mappability, and linear group actions cannot be weakly pseudorandom without impractical restrictions.

Inspired by Moriya *et al.* *SIGAMAL* encryption [37], Leroux and Roméas [34] circumvented the mappability requirement by introducing the *Triple Orbital Group Action* (TOGA) abstraction, and instantiated the *TOGA-UE* scheme from isogeny-based group actions. Among post-quantum UE schemes, *TOGA-UE* offers the shortest ciphertexts—just an elliptic curve and a single point—and requires only one isogeny computation per encryption, update, or decryption. Despite this efficiency, this scheme is malleable and thus not CCA-secure.

Recently, Meers and Riepel [36] proposed *COM-UE*, the first CCA-secure post-quantum UE scheme. Derived from *GAINE* [34] using bitwise encryption

and the Tag-then-Encrypt paradigm, COM-UE achieves IND-UE-CCA security in the Algebraic Group Action Model [21] under an ad hoc assumption implied by the Discrete Logarithm with Auxiliary Input [3]. This resolves the open problem of constructing a CCA-secure post-quantum UE scheme, but at a steep cost: each encryption, decryption, and update requires one isogeny computation per plaintext and integrity tag bit. An overview of existing UE schemes, their hardness assumptions, and achieved security notions is given in Table 1.

Family	Scheme	Security (IND)	Assumption	Model
Discrete Log	RISE [30]	(rand, UE, CPA)	DDH	Standard
	E&M [28]	(det, ENC/UPD, CCA)	DDH	ROM
	SHINE0 [8]	(det, UE, CCA)	DDH, CDH	Ideal Cipher
Pairings	NYUAE [28]	(rand, ENC/UPD, RCCA)	SXDH	Standard
	SS23 [43]	(rand, UE, CPA) ⁺	SXDH	Standard
Lattices	Jia20 [25]	(rand, UE, CPA)	LWE	Standard
	Nis22 [38]	(rand, UE, CPA)	LWE	Standard
	GP23 [26]	(rand, UE, CPA)	LWE	Standard
Isogenies	GAINE0* [34]	(det, UE, CCA)	wk-PR, wk-UP	Ideal Cipher
	TOGA-UE [34]	(det, UE, CPA)	P-CSSDDH	Standard
	BIN-UE [36]	(det, UE, CPA)	wk-PR	Standard
	COM-UE [36]	(det, UE, CCA)	wk-PR, DLAI	ROM + AGAM
	DINE (this work)	(det, UE, CCA)	PP,UP,DIPHTI	Ideal Cipher

* no secure instantiation known, ⁺ satisfies a stronger definition with expiry epochs

Table 1. Overview of existing ciphertext-independent UE constructions updated from [36] with our new construction DINE. We use **rand** for schemes with randomized updates and **det** for deterministic ones. Our new assumptions PP, UP, and DIPHTI are presented and studied in Section 4.

Metric	KeyGen/Update	Key Enc	Dec	Upd
Time (ms)	1121	48	86	28

Table 2. Average running time (over 10 runs) of the C implementation of DINE on an Intel Core i7 at 2.3GHz with turbo-boost disabled, adapted from the latest SQIsign code for the NIST submission <https://sqisign.org>.

Overview of the contributions. We introduce DINE, a new isogeny-based post-quantum UE scheme achieving **detIND-UE-CCA** security while requiring only a small number of power-of-2 isogeny evaluations in dimension 2 for each encryption, decryption, and ciphertext update. Its security relies on several new computational assumptions related to well-studied problems in isogeny-based cryptography. At NIST security level 1, DINE encrypts 28B messages with ciphertexts of 320B and update tokens of 896B. For the same message size, the CSIDH-512 instantiation of the only other CCA-secure post-quantum UE scheme, COM-UE [36], features ciphertexts and tokens of size 22,528B. Assuming a generous estimate of 10ms per group action computation for [36], each COM-UE encryption and update would require around 3.5s.

We further demonstrate the practicality of DINE through a full C implementation, whose performance results are reported in Table 2. At NIST security

level 1, updates take only 28ms, encryptions 48ms, and decryptions 86ms. To the best of our knowledge, this is the first implementation of a post-quantum ciphertext-independent UE scheme reported in the literature.

To achieve this, we follow the recent trend [17,5,33,4] in isogeny-based cryptography of building cryptographic schemes from arbitrary degree isogenies by using two powerful tools: the high-dimensional (HD) isogeny representation stemming from Kani’s Lemma, and the Deuring correspondence.

We also introduce several new algorithms around the Deuring correspondence that may be of independent interest, as well as novel computational problems that could serve as a foundation for future cryptographic constructions.

Technical overview. We now provide an overview of the main technical ideas behind our construction. The detailed description can be found in Section 3.2.

The basic principle behind our encryption mechanism is as follows: given a secret isogeny of public domain, messages can be encrypted by mapping them invertibly to points of the domain, and then evaluating the secret isogeny on these points. The existence of a dual isogeny, that acts almost as an inverse, gives the means to decrypt.

This symmetric encryption scheme can be made updatable by computing tokens as alternate paths between the codomains of two secret keys. A ciphertext is updated by pushing its components (which are image points) through this token isogeny. Such a token can be efficiently computed if one knows the quaternion ideals corresponding to the secret isogenies via the Deuring correspondence, using for instance the KLPT algorithm [29] and the ideal-to-isogeny machinery at the core of the SQIsign signature scheme and its variants [20,17,5].

The only problem is that the resulting scheme is not secure. Indeed, isogenies act linearly on points, and thus any linear relation between some ciphertexts is preserved under updates. In the quantum setting, this leads to an attack against the IND-UE property, since three points always admit a linear relation, which can be efficiently found in quantum polynomial time using Shor’s algorithm. To prevent this attack, we replace points with isogenies. Indeed, every subgroup of an elliptic curve defines the kernel of some isogeny, yielding a one-to-one correspondence between isogenies and their kernels. In most cases, one can “evaluate”⁴ an isogeny through another isogeny by computing the image of its kernel. Thus, if messages can be mapped to isogenies invertibly, we can translate our previous scheme into a UE scheme where ciphertexts are isogenies instead of points. If it is possible to go from the ciphertext isogenies to their kernels efficiently, then the security of this new scheme is no better than the previous one. However, if the ciphertext isogenies are chosen so that even representing a point of their kernel is infeasible, then our scheme appears resistant to known attacks.

This property is achieved by using isogenies of large prime degree N over a field of characteristic p such that p has large order in $\mathbb{F}_N^*$. That way, the points of order N (which constitute the kernel of isogenies of degree N) are defined over an extension field of degree roughly N , making explicitly representing a kernel point infeasible. For random large primes p and N this will be true with over-

⁴ this is called a pushforward

whelming probability. Recent progress in isogeny-based cryptography has made it possible to efficiently handle and represent large prime degree isogenies using high-dimensional isogenies [17,5], enabling their practical use in our scheme.

Thus, there is some hope for security, but a key challenge remains: mapping messages to isogenies in an invertible way. Unlike mapping messages to points, this is nontrivial for isogenies. Notably, the kernels of our ciphertext isogenies are deliberately hard to compute, so mapping through them is infeasible. Moreover, computing the HD representation of an isogeny without prior knowledge of an efficient representation is hard, ruling out a mapping via this representation.

To address this challenge, we rely on the Deuring correspondence. Indeed, mapping messages to ideals invertibly is straightforward, and using the ideal-to-isogeny algorithm from [5], one can efficiently compute their HD representation, enabling practical encryption. Decryption, however, requires inverting this mapping, *i.e.*, recovering the ideal from the HD representation. While a quantum polynomial-time algorithm exists for this task [11,13], no efficient classical algorithm is known—in fact, this computational problem was even used to build the pSIDH key exchange protocol [31].

Our final idea to overcome this obstacle is to use endomorphisms instead of individual isogenies. Indeed, an endomorphism of degree N^2 can be decomposed into two isogenies of degree N between the same pair of curves. This allows us to construct our scheme as before, but with each ciphertext consisting of two degree N isogenies instead of one. Every endomorphism of a curve can be represented by four integers corresponding to its decomposition in a basis of the endomorphism ring. Crucially, these integers can be efficiently recovered by evaluating the endomorphism on points of sufficiently large order to compute trace pairings with the basis elements (see, *e.g.*, the CheckTrace algorithm [31, Algorithm 9]). Hence, the integers can be extracted from the HD representations—composed of evaluated points—of the two degree N isogenies forming the endomorphism, enabling efficient inversion of the HD representation back to the endomorphism.

If the endomorphism ring is well-chosen, a random string can be mapped to four integers representing an endomorphism of norm N^2 . For instance, in the public domain with j -invariant 1728, the endomorphism ring contains a suborder with basis $1, i, j, k$, and the norm of an endomorphism $x + iy + jz + kt$ is $x^2 + y^2 + p(z^2 + t^2)$. Our idea is to map a message together with a small random string to integers $z, t \leq \sqrt{N^2/2p}$. For each choice of z, t , suitable integers x, y can be efficiently computed using Cornacchia’s algorithm with probability roughly $1/\log p$. By trying $O(\log p)$ random strings, one can construct an endomorphism $\theta = x + iy + jz + kt$ to encrypt the message. As discussed above, the integers z, t can then be recovered efficiently from the two degree N isogenies composing θ , providing all the ingredients needed to implement our UE scheme.

The security of our scheme relies on several new problems related to the following assumption in characteristic p : the pushforwards of two degree N isogenies through a secret isogeny are computationally indistinguishable from two random degree N isogenies, provided N and p are chosen such that the N -torsion points of supersingular elliptic curves in characteristic p cannot be efficiently computed.

We conclude our overview by explaining how we efficiently “evaluate” secret and token isogenies on ciphertext isogenies using the HD representation. In particular, Kani’s Lemma—the key ingredient of the HD representation—enables the pushforward of an N -isogeny through a $(2^f - N)$ -isogeny by computing a 2^f -isogeny in dimension 2. Consequently, secret key and token isogenies of degree $(2^f - N)^k$ can be efficiently used by sequentially applying Kani’s Lemma k times to push N -isogenies through $(2^f - N)^k$ -isogenies.

The exponent k may differ between keys and tokens. Taking a smaller k improves efficiency. Taking $k = 1$ would be ideal but it would not produce a uniformly random isogeny when looking at the isomorphism class of the image on the N -torsion subgroups. To get close to uniform distribution, we need heuristically to take at least $k = 4$ (more details on that in Section 4). However, finding isogenies of prescribed norm between two curves is generally hard, and known algorithms such as KLPT (and variants [29,20]) require the norm to be sufficiently large. As a result, it may be necessary to choose $k > 4$ for either keys or tokens—for instance, one could take $k = 4$ for keys, but then $k > 4$ is required for tokens, or vice versa.

Accordingly, we consider two variants of our scheme: one with $k = 4$ for tokens, optimizing update speed, and one with $k = 4$ for secret keys, optimizing encryption and decryption speed. We prioritize the former, as update efficiency is more critical and the latter is harder to analyze rigorously.

Outline of the paper. Section 2 reviews UE and isogeny background; Section 3 presents DINE and Section 4 analyzes its security; Section 5 details the algorithmic instantiation, and Section 6 covers its implementation.

2 Preliminaries

Notations. We write $s \leftarrow \mathcal{D}$ to denote sampling from a distribution $\mathcal{D}$. An algorithm $\mathcal{A}$ with oracle access to or is denoted $\mathcal{A}^{\text{or}}$. For a function f depending (possibly implicitly) on a parameter λ , we write $f = \text{negl}(\lambda)$ when $f = o(\lambda^{-n})$ for all $n \in \mathbb{N}$. Throughout, $\mathcal{O}_0$ denotes the special maximal order in the quaternion algebra $B_{p,\infty}$ defined in Section 2.2. For pushforwards of ideals or isogenies (see Section 2.2), we use the shorthand $[I]_*(J, K) := ([I]_*J, [I]_*K)$.

2.1 Updatable Encryption

In this section, we describe the syntax and security definitions of UE, following the presentations of [38,8,30,25]. An UE scheme operates in *epochs*, where each epoch corresponds to an index incremented with every key update. For proof purposes, let $n + 1$ denote the maximum number of epochs.

Definition 1. *An updatable encryption scheme UE for message space $\mathcal{M}$ consists of a tuple of probabilistic polynomial time algorithms (UE.Setup, UE.KeyGen, UE.TokenGen, UE.Enc, UE.Dec, UE.Upd) where:*

- $\text{UE.Setup}(1^\lambda) \rightarrow \text{pp}$: The setup algorithm takes as input the security parameter and outputs a public parameter pp .
- $\text{UE.KeyGen}(\text{pp}) \rightarrow k_e$: The key generation algorithm takes as input the public parameter pp and outputs an epoch key k_e .
- $\text{UE.Enc}(k, m) \rightarrow c$: The encryption algorithm takes as input an epoch key k , a message m , and outputs a ciphertext c .
- $\text{UE.Dec}(k, c) \rightarrow m$: The decryption algorithm takes as input an epoch key k , a ciphertext c , and outputs a message m or $\perp$.
- $\text{UE.TokenGen}(k_e, k_{e+1}) \rightarrow \Delta_{e+1}$: The token generation algorithm takes as input two keys of consecutive epochs e and $e+1$ and outputs a token Δ_{e+1} .
- $\text{UE.Upd}(\Delta_{e+1}, c_e) \rightarrow c_{e+1}$: The update algorithm takes as input a token Δ_{e+1} , a ciphertext c_e , and outputs a ciphertext c_{e+1} .

Definition 2 (Correctness). For any $m \in \mathcal{M}$, for $0 \leq e_1 \leq e_2 \leq n+1$, it holds that $\Pr[\text{UE.Dec}(k_{e_2}, c_{e_2}) \neq m] \leq \text{negl}(\lambda)$, where $\text{pp} \leftarrow \text{UE.Setup}(1^\lambda)$, $k_{e_1}, \dots, k_{e_2} \leftarrow \text{UE.KeyGen}(\text{pp})$, $c_{e_1} \leftarrow \text{UE.Enc}(k_{e_1}, m)$, and $\Delta_{i+1} \leftarrow \text{UE.TokenGen}(k_i, k_{i+1})$, $c_{i+1} \leftarrow \text{UE.Upd}(\Delta_{i+1}, c_i)$ for $i \in [e_1, e_2 - 1]$.

Security definitions. In all our UE schemes, the Upd algorithm is *deterministic*. We therefore consider only security definitions in the deterministic update setting. An adaptation of the usual indistinguishability security notions for UE is given in Definition 3. We briefly explain the oracles used in this definition; they are formally defined in Fig. 2. The Enc and Dec oracles implement standard encryption and decryption queries. The Next oracle increments the epoch counter and samples a new secret key along its associated token; if a challenge ciphertext exists, it is updated accordingly. The Corr oracle, on input key (resp. token) and an epoch, returns the corresponding key (resp. token). The Chall oracle samples a uniform bit b and returns either an encryption of the input message or an update of the input ciphertext, depending on b . The returned ciphertext is the challenge, and the adversary's goal is to guess b . Finally, the Upd $\check{C}$ oracle returns the updated version of the challenge ciphertext. All these oracles track the values returned to, or computable by, the adversary; these values are stored in the sets denoted in calligraphic font (see Section 2.1 for details).

Definition 3 (detIND-UE-atk [8]). Let $\text{UE} = (\text{UE.Setup}, \text{UE.KeyGen}, \text{UE.TokenGen}, \text{UE.Enc}, \text{UE.Dec}, \text{UE.Upd})$ be an updatable encryption scheme. The detIND-UE-atk advantage, for $\text{atk} \in \{\text{CPA}, \text{CCA}\}$, of an adversary $\mathcal{A}$ against UE is given by

$$\text{Adv}_{\text{UE}}^{\text{detIND-UE-atk}}(\mathcal{A}) := |\Pr[\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{detIND-UE-atk-0}} = 1] - \Pr[\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{detIND-UE-atk-1}} = 1]|$$

where the confidentiality experiment $\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{detIND-UE-atk-}b}$ is given in Fig. 1.

Next, in the ciphertext integrity (CTXT) game, the adversary is given access to the oracles $\mathcal{O}.\text{Enc}$, $\mathcal{O}.\text{Next}$, $\mathcal{O}.\text{Upd}$, and $\mathcal{O}.\text{Corr}$. At some point, $\mathcal{A}$ attempts to provide a ciphertext forgery via the oracle $\mathcal{O}.\text{Try}$ defined in Fig. 1. The adversary wins if the forgery is valid, *i.e.*, if it decrypts to a message rather than $\perp$. The INT-CTXT^s advantage is defined in Definition 4.

$\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{detIND-UE-atk-}b}(\lambda)$	$\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{INT-CTXT}^S}(\lambda)$
Setup (1^λ) $\text{ors} \leftarrow \mathcal{O}.\{\text{Enc}, \text{Upd}, \text{Next}, \text{Corr}\}$ if $\text{atk} = \text{CCA}$ then $\text{ors} \leftarrow \text{ors} \cup \{\mathcal{O}.\text{Dec}\}$ $(\bar{M}, \bar{C}) \leftarrow \mathcal{A}^{\text{ors}}(1^\lambda)$; $\text{phase} \leftarrow 1$; $\tilde{e} \leftarrow e$ $\tilde{C}_e \leftarrow \mathcal{O}.\text{Chall}(\bar{M}, \bar{C})$ $b' \leftarrow \mathcal{A}^{\text{ors}, \mathcal{O}.\text{Upd}\tilde{C}}(\tilde{C}_e)$ if $\mathcal{K}^* \cap \mathcal{C}^* \neq \emptyset$ or $\mathcal{I}^* \cap \mathcal{C}^* \neq \emptyset$ then $\text{twf} \leftarrow 1$ if $\text{twf} = 1$ then $b' \xleftarrow{\$} \{0, 1\}$ return b'	Setup (1^λ) $\text{win} \leftarrow 0$ $\mathcal{A}^{\mathcal{O}.\text{Enc}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{Upd}, \mathcal{O}.\text{Corr}, \mathcal{O}.\text{Try}}(\lambda)$ if $\text{twf} = 1$ then $\text{win} \leftarrow 0$ return win $\mathcal{O}.\text{Try}(\tilde{C})$
if $\text{phase} = 1$ then return $\perp$ $\text{phase} \leftarrow 1$ if $(e \in \mathcal{K}^* \text{ or } \tilde{C} \in \mathcal{L}^*)$ then $\text{twf} \leftarrow 1$ $M \text{ or } \perp \leftarrow \text{UE}.\text{Dec}(k_e, \tilde{C})$ if $M \neq \perp$ then $\text{win} \leftarrow 1$	

Fig. 1. Description of $\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{detIND-UE-atk-}b}$, $\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{INT-CTXT}^S}$ and oracle $\mathcal{O}.\text{Try}$ for UE scheme UE (with deterministic updates) and adversary $\mathcal{A}$, for $\text{atk} \in \{\text{CPA}, \text{CCA}\}$. Oracles are given in Fig. 2. Trivial win conditions are discussed in Section 2.1.

Setup (1^λ)	$\mathcal{O}.\text{Enc}(M)$	$\mathcal{O}.\text{Dec}(C)$
$\text{pp} \leftarrow \text{UE}.\text{Setup}(1^\lambda)$	$C \leftarrow \text{UE}.\text{Enc}(k_e, M)$	if $\text{phase} = 1$ and $C \in \tilde{\mathcal{L}}$
$k_0 \leftarrow \text{UE}.\text{KeyGen}(\text{pp})$	$c \leftarrow c + 1$	$\text{twf} \leftarrow 1$
$\Delta_0 \leftarrow \perp$; $e, c \leftarrow 0$	$\mathcal{L} \leftarrow \mathcal{L} \cup \{(c, C, e)\}$	$M \text{ or } \perp \leftarrow \text{UE}.\text{Dec}(k_e, C)$
$\text{phase}, \text{twf} \leftarrow 0$	return C	return $M \text{ or } \perp$
$\mathcal{L}, \tilde{\mathcal{L}}, \mathcal{C}, \mathcal{K}, \mathcal{T} \leftarrow \emptyset$		
$\mathcal{O}.\text{Next}$	$\mathcal{O}.\text{Upd}(C_{e-1})$	$\mathcal{O}.\text{Corr}(\text{inp}, \hat{e})$
$e \leftarrow e + 1$	if $(j, C_{e-1}, e - 1) \notin \mathcal{L}$	if $\hat{e} > e$ then return $\perp$
$k_e \leftarrow \text{UE}.\text{KeyGen}(\text{pp})$	return $\perp$	if $\text{inp} = \text{key}$
$\Delta_e \leftarrow \text{UE}.\text{TokenGen}(k_{e-1}, k_e)$	$C_e \leftarrow \text{UE}.\text{Upd}(\Delta_e, C_{e-1})$	$\mathcal{K} \leftarrow \mathcal{K} \cup \{\hat{e}\}$
if $\text{phase} = 1$	$\mathcal{L} \leftarrow \mathcal{L} \cup \{(j, C_e, e)\}$	return $k_{\hat{e}}$
$\tilde{C}_e \leftarrow \text{UE}.\text{Upd}(\Delta_e, \tilde{C}_{e-1})$	return C_e	if $\text{inp} = \text{token}$
		$\mathcal{T} \leftarrow \mathcal{T} \cup \{\hat{e}\}$
		return $\Delta_{\hat{e}}$
$\mathcal{O}.\text{Chall}(\bar{M}, \bar{C})$	$\mathcal{O}.\text{Upd}\tilde{C}$	
if $\text{phase} \neq 1$ then return $\perp$	if $\text{phase} \neq 1$ then return $\perp$	
if $(\cdot, \bar{C}, e - 1) \notin \mathcal{L}$ then return $\perp$	$\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$	
if $b = 0$ then $\tilde{C}_e \leftarrow \text{UE}.\text{Enc}(k_e, \bar{M})$	$\tilde{\mathcal{L}} \leftarrow \tilde{\mathcal{L}} \cup \{(\tilde{C}_e, e)\}$	
else $\tilde{C}_e \leftarrow \text{UE}.\text{Upd}(\Delta_e, \bar{C})$	return $\tilde{C}_e$	
$\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$; $\tilde{\mathcal{L}} \leftarrow \tilde{\mathcal{L}} \cup \{(\tilde{C}_e, e)\}$		
return $\tilde{C}_e$		

Fig. 2. Oracles in security games for UE with deterministic updates. Computing the leakage sets is discussed in Section 2.1.

Definition 4 ([8]). Let $\text{UE} = \{\text{UE.KeyGen}, \text{UE.TokenGen}, \text{UE.Enc}, \text{UE.Dec}, \text{UE.Upd}\}$ be an UE scheme. The INT-CTXT^s advantage of an adversary $\mathcal{A}$ against UE is defined as $\text{Adv}_{\text{UE}}^{\text{INT-CTXT}^s}(\mathcal{A}) := \Pr[\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{INT-CTXT}^s} = 1]$ where the experiment $\text{Exp}_{\text{UE}, \mathcal{A}}^{\text{INT-CTXT}^s}$ is given in Fig. 1.

Leakage sets. We follow the bookkeeping technique [30,8] to maintain the epoch leakage sets.

- $\mathcal{C}$: Set of epochs in which the adversary obtained an updated version of the challenge ciphertext (via $\mathcal{O}.\text{Chall}$ or $\mathcal{O}.\text{Upd}\tilde{\mathcal{C}}$).
- $\mathcal{K}$: Set of epochs in which the adversary corrupted the encryption key.
- $\mathcal{T}$: Set of epochs in which the adversary corrupted the update token.

In addition, the adversary may learn ciphertexts and their updates:

- $\mathcal{L}$: Set of non-challenge ciphertexts (from $\mathcal{O}.\text{Enc}$ or $\mathcal{O}.\text{Upd}$), containing tuples (c, C, e) where c is a counter incremented with each $\mathcal{O}.\text{Enc}$ query.
- $\tilde{\mathcal{L}}$: Set of updated versions of the challenge ciphertext (generated by $\mathcal{O}.\text{Next}$ and returned by $\mathcal{O}.\text{Upd}\tilde{\mathcal{C}}$), containing tuples $(\tilde{C}, e)$.

Trivial wins via keys and ciphertexts. We define extended epoch leakage sets $\mathcal{C}^*$, $\mathcal{K}^*$, and $\mathcal{T}^*$ derived from $\mathcal{C}$, $\mathcal{K}$, and $\mathcal{T}$. These extended sets capture situations where the adversary obtains both a challenge ciphertext update and the means to decrypt it. In particular, if $\mathcal{C}^* \cap \mathcal{K}^* \neq \emptyset$, then there exists an epoch in which the adversary knows both the epoch key and a valid update of the challenge ciphertext, resulting in a trivial win. The challenger computes these extended sets after the adversary halts, with precise definitions given in Appendix A.

Trivial wins via direct updates. We define $\mathcal{I}$ as the set of epochs in which the adversary learned an updated version of the challenge input ciphertext $\tilde{C}$. Its extension $\mathcal{I}^*$ also accounts for information inferred through token corruption.

Since Upd is deterministic, an updated ciphertext is uniquely determined by a token and a ciphertext. Hence, if $\mathcal{I}^* \cap \mathcal{C}^* \neq \emptyset$, the adversary trivially wins: there exists an epoch in which the adversary knows both an updated version of $\tilde{C}$ and a challenge-equal ciphertext, and comparing them suffices to win.

Trivial wins in ciphertext integrity games. An adversary who corrupts an epoch key can trivially forge ciphertexts in that epoch. We exclude this case by setting $\text{twf} = 1$ whenever a forgery is attempted in an epoch contained in $\mathcal{K}^*$.

Another trivial win arises if the adversary knows a ciphertext $(C, e_1) \in \mathcal{L}$ together with all tokens from epoch $e_1 + 1$ up to some epoch e_2 . In this case, it can simply update C to epoch e_2 and present it as a valid forgery. To rule this out, we extend $\mathcal{L}$ to a set $\mathcal{L}^*$ that also accounts for ciphertexts derived via token corruption. If $\mathcal{O}.\text{Try}$ receives a ciphertext from $\mathcal{L}^*$, it sets $\text{twf} = 1$. The algorithm of [8] used to compute $\mathcal{L}^*$ during the game is recalled in Fig. 8 of Appendix A.

Composition result for CPA, CTXT and CCA security. We conclude this section by recalling a generic composition result of Boyd *et al.* [8, Theorem 3], which relates CPA security and ciphertext integrity to CCA security for UE schemes.

Theorem 1 ([8]). *For any detIND-UE-CCA adversary $\mathcal{A}$ against an UE scheme UE, there exist a INT-CTXT^s adversary $\mathcal{B}$ and a detIND-UE-CPA adversary $\mathcal{B}'$ such that $\text{Adv}_{\text{UE}}^{\text{detIND-UE-CCA}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{UE}}^{\text{INT-CTXT}^s}(\mathcal{B}) + \text{Adv}_{\text{UE}}^{\text{detIND-UE-CPA}}(\mathcal{B}')$.*

2.2 Isogenies

We assume familiarity with elliptic curves and isogenies up to the level of [22], and recall the necessary background in App. C. For background on quaternion algebras and the Deuring correspondence, see [45]. We also provide a self-contained summary in App. D, adapted from [46].

Below, we recall useful conventions and notions. Let p be a prime congruent to 3 mod 4. Then $B_{p,\infty} = B(-1, -p)$ [40], where $B_{p,\infty}$ denotes the unique quaternion algebra ramified exactly at p and ∞ . In this case, $B_{p,\infty}$ contains the maximal order $\mathcal{O}_0 := \langle 1, i, \frac{1+i}{2}, i\frac{1+i}{2} \rangle$. Throughout this paper, p is always a prime congruent to 3 mod 4, and $\mathcal{O}_0$ always denotes this specific maximal order.

Commutative isogeny diagrams. Let E, E', E'' be elliptic curves and $\varphi : E \rightarrow E'$, $\psi : E \rightarrow E''$ two separable isogenies of coprime degrees d_1 and d_2 . Then there exists a fourth elliptic curve E''' together with two *pushforward isogenies* $[\varphi]_*\psi$ and $[\psi]_*\varphi$ going from E' and E'' to E''' and satisfying $\ker([\varphi]_*\psi) = \varphi(\ker(\psi))$, $\deg([\varphi]_*\psi) = d_2$ and $\ker([\psi]_*\varphi) = \psi(\ker(\varphi))$, $\deg([\psi]_*\varphi) = d_1$. This gives rise to the commutative diagram in Fig. 3, where both paths correspond to two decompositions of the same isogeny $\Phi = [\psi]_*\varphi \circ \psi = [\varphi]_*\psi \circ \varphi$.

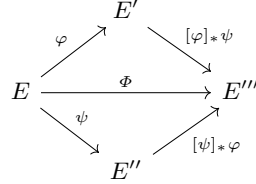


Fig. 3. A commutative isogeny diagram.

The dual notion of *pullback isogenies* is defined as follows: given $\varphi : E \rightarrow E'$ and $\rho : E' \rightarrow E'''$ of coprime degrees, the pullback of ρ by φ is $[\varphi]^*\rho := [\hat{\varphi}]_*\rho$.

Efficient high-dimensional isogeny representation. Building on insights from SIDH attacks, an isogeny $\varphi : E \rightarrow E'$ can be efficiently represented using the images of a basis P, Q of $E[2^f]$, provided $\deg \varphi < 2^f$. The most efficient known method employs dimension 2 isogenies and evaluates P, Q via an auxiliary isogeny $\psi : E \rightarrow E''$ of degree $2^f - \deg \varphi$. The main theoretical result behind this way of representing isogenies is called Kani's Lemma [27].

Theorem 2 (Kani's Lemma, [27]). *Let d_1 and d_2 be two coprime positive integers. Given a (d_1, d_2) -isogeny commutative diagram as in Fig. 3, where $[\varphi]_*\psi$ is denoted by ψ' and $[\psi]_*\varphi$ is denoted by φ' , the isogeny $\Psi : E \times E''' \rightarrow E' \times E''$ corresponding to the matrix*

$$\begin{pmatrix} \varphi & \hat{\psi}' \\ -\psi & \hat{\varphi}' \end{pmatrix}$$

is a $(d_1 + d_2)$ -isogeny between these products of elliptic curves with their principal product polarisation. Moreover, the kernel of Ψ is

$$\ker \Psi = \{(\hat{\varphi}(R), \psi'(R)) \mid R \in E'[d_1 + d_2]\}$$

It is then routine to verify that $\ker \Psi = \{(\deg \varphi)R, (\psi' \circ \varphi)(R) \mid R \in E[2^f]\}$. Hence, knowing $\varphi(P)$ and $\varphi(Q)$, i.e. the action of φ on the 2^f -torsion of E , together with the ability to construct an arbitrary isogeny $\psi' : E' \rightarrow E'''$ of degree $2^f - \deg \varphi$, suffices to compute Ψ . By [18], Ψ can then be efficiently evaluated on any point of $E \times E'''$. In particular, since $\Psi((R, 0_{E'''})) = (\varphi(R), -\psi(R))$ for any $R \in E$, we can efficiently evaluate φ everywhere. This observation motivates the following definition of a *high-dimensional (HD) representation* of an isogeny φ .

Definition 5. Let $\varphi : E \rightarrow E'$ be an isogeny of (implicitly known) degree $N < 2^f$. An HD representation of φ is a tuple $s_{\text{HD}}(\varphi) := E, P, Q, E', \varphi(P), \varphi(Q)$ where P, Q form a basis of $E[2^f]$. If the points P, Q can be deterministically generated, they are called a canonical basis of $E[2^f]$. In this case, P and Q can be omitted from $s_{\text{HD}}(\varphi)$ and we write $s_{\text{HD}}^{\text{can}}(\varphi) := E, E', \varphi(P), \varphi(Q)$.

3 UE from the Deuring Correspondence

Parameters. We consider a finite field of characteristic p of the form $c2^f - 1$, where the cofactor c is chosen as small as possible. This choice ensures that the 2^f -torsion of a supersingular curve E can be defined over $\mathbb{F}_{p^2}$.

Additionally, we select a prime $N < 2^f$ such that p has maximal order in $\mathbb{F}_N^*$. This guarantees that the N -torsion points of elliptic curves lie in a field of maximal degree, which is crucial for security.

3.1 Algorithmic Building Blocks

In this section, we list the key algorithmic building blocks used in our construction. For now, these algorithms are treated as black boxes; their precise instantiation is provided and explained in Section 5.

- **EvalAnyIsoCanBasis**: given an ideal I corresponding to an isogeny $\varphi_I : E \rightarrow E_I$, outputs its canonical HD representation $s_{\text{HD}}^{\text{can}}(\varphi_I)$.
- **HDPushForward**: given $s_{\text{HD}}(\varphi_1), s_{\text{HD}}(\varphi_2)$ for two isogenies φ_1, φ_2 with the same domain and respective degrees N and $2^f - N$, outputs $s_{\text{HD}}([\varphi_2]_* \varphi_1)$.
- **ToCanonicalRepresentation**: converts an HD representation $s_{\text{HD}}(\varphi)$ into its canonical form $s_{\text{HD}}^{\text{can}}(\varphi)$.
- **PartialRepresentInteger_{N2}**: given integers z, t , outputs either $\perp$ or integers x, y such that $x^2 + y^2 + p(z^2 + t^2) = N^2$.
- **IdealPullBack**: given an ideal I corresponding to $\varphi_I : E \rightarrow E_I$ and $s_{\text{HD}}^{\text{can}}(\psi)$ with $\psi : E_I \rightarrow E$ of degree N , outputs $s_{\text{HD}}^{\text{can}}([\varphi_I]^* \psi)$.

- **HalfEndoToCoord**: given $s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$ for two isogenies with the same domain E_0 and isomorphic codomain, outputs (when possible) the four coordinates x, y, z, t such that the endomorphism $\hat{\varphi}_2 \circ \varphi_1$ corresponds via the Deuring correspondence to the principal ideal $\mathcal{O}_0(x + iy + jz + kt)$.
- **SpecialKLPT** _{$2^f - N$} : given an ideal I , computes an equivalent ideal $J \sim_{\mathbb{Z} + N\mathcal{O}_L(I)}$ I of norm $(2^f - N)^k$ for some $k \in \mathbb{N}$.

3.2 The DINE Scheme

In this section, we present our main scheme, called DINE (Deuring Ideal cipher Nonce-based Encryption). As noted in the technical overview, there is a trade-off between the efficiency of **Enc/Dec** and that of **Upd**. We choose to prioritize faster updates, which requires selecting the update token before generating the updated secret key. Consequently, we slightly modify the standard UE interface. Instead of the usual **TokenGen** algorithm, which takes two keys k_e, k_{e+1} generated via **KeyGen** and outputs the token Δ_{e+1} , we introduce an **UpdateKey** algorithm. **UpdateKey** takes only k_e as input and outputs both k_{e+1} and Δ_{e+1} . This modification does not affect the overall functionality of the UE scheme.

Remark 1. In Appendix E, we describe a second variant of our scheme, which prioritizes faster **Enc** and **Dec**. In this variant, since the updated keys are generated before the update tokens, it adheres to the standard UE framework.

We assume the existence of an invertible map $\pi : \{0, 1\}^{n(\lambda) + m(\lambda)} \rightarrow [0, m]^2$ with $m = N/\sqrt{2p}$, where $\{0, 1\}^{n(\lambda)}$ is the message space and $\{0, 1\}^{m(\lambda)}$ is the nonce space. A detailed description of our scheme is provided in Fig. 4. For clarity, we also include diagrams illustrating the main operations: key generation and rotation in Fig. 5, encryption in Fig. 6, and ciphertext updates in Fig. 7.

We first prove the correctness of ciphertext updates, and then the correctness of DINE as a whole.

Proposition 1 (Correctness of updates). *Let k_e be a key corresponding to an ideal I_e , and define $C^{(e)} := ([\varphi_{I_e}]_* \varphi_{I_1}, [\varphi_{I_e}]_* \varphi_{I_2})$ for ideals $I_1 := \mathcal{O}_0\langle\theta_{N,M}, N\rangle$ and $I_2 := \mathcal{O}_0\langle\overline{\theta_{N,M}}, N\rangle$, where $N \in \mathbb{N}$ and $\theta_{N,M}$ is a quaternion with $\text{Nrd}(\theta_{N,M}) = N^2$. If $k_{e+1}, \Delta_{e+1} := \text{UpdateKey}(k_e)$, then $\text{Upd}(\Delta_{e+1}, C^{(e)}) = ([\varphi_{I_{e+1}}]_* \varphi_{I_1}, [\varphi_{I_{e+1}}]_* \varphi_{I_2})$ where I_{e+1} is the ideal corresponding to k_{e+1} under the Deuring correspondence.*

Proof. We prove correctness for the first component of the ciphertext; the argument for the second component is analogous. Retaining the notations of the proposition, let J_{e+1} denote the ideal corresponding to the token isogeny Δ_{e+1} .

It suffices to show that, for any isogeny ψ of degree N with domain E_0 , we have $[\Delta_{e+1} \circ \varphi_{k_e}]_* \psi = [\varphi_{k_{e+1}}]_* \psi$. Once this equality holds, we obtain

$$C_1^{(e+1)} := [\Delta_{e+1}]_* C_1^{(e)} = [\Delta_{e+1}]_* ([\varphi_{k_e}]_* \varphi_{I_1}) = [\Delta_{e+1} \circ \varphi_{k_e}]_* \varphi_{I_1} = [\varphi_{k_{e+1}}]_* \varphi_{I_1}$$

which establishes the correctness of the update for any choice of φ_{I_1} and thus proves the proposition.

KeyGen(pp)	Enc(k_e, M)
$I \leftarrow \text{elt. of a random } (\mathbb{Z} + N\mathcal{O}_0)\text{-ideal class.}$ $I \leftarrow \text{SpecialKLPT}_{2^f - N}(I)$ return I	do Generate a random nonce R $(z, t) \leftarrow \pi(R \ M)$ $s \leftarrow \text{PartialRepresentInteger}_{N^2}(z, t)$
UpdateKey (k_e)	while $s = \perp$
Parse k_e as an ideal J_{e+1}^0 For $1 \leq i \leq 4$: $J_{e+1}^i \leftarrow \text{random } \mathcal{O}_R(J_{e+1}^{i-1})\text{-ideal}$ of norm $2^f - N$ $s_{\text{HD}}^{\text{can}}(\delta_{e+1}^i) \leftarrow \text{EvalAnyIsoCanBasis}(J_{e+1}^i)$ $\Delta_{e+1} = \left(s_{\text{HD}}^{\text{can}}(\delta_{e+1}^i) \right)_{1 \leq i \leq 4}$ $J_{e+1} \leftarrow \prod_{i=1}^4 J_{e+1}^i$ $I_{e+1} \leftarrow \text{SpecialKLPT}_{2^f - N}(I_e \cdot J_{e+1})$ return Δ_{e+1}, I_{e+1}	Parse s as two integers x, y $\theta_{N,M} \leftarrow x + iy + jz + kt$ $I_1 \leftarrow \mathcal{O}_0 \langle \theta_{N,M}, N \rangle; I_2 \leftarrow \mathcal{O}_0 \langle \overline{\theta_{N,M}}, N \rangle$ $I_1 \leftarrow [k_e]_* I_1; I_2 \leftarrow [k_e]_* I_2$ $s_1 := s_{\text{HD}}^{\text{can}}(\varphi_{I_1}) \leftarrow \text{EvalAnyIsoCanBasis}(I_1)$ $s_2 := s_{\text{HD}}^{\text{can}}(\varphi_{I_2}) \leftarrow \text{EvalAnyIsoCanBasis}(I_2)$ return $C_e := (s_1, s_2)$
Upd (Δ_{e+1}, C_e)	Dec (k_e, C_e)
Parse Δ_{e+1} as $\left(s_{\text{HD}}^{\text{can}}(\delta_{e+1}^i) \right)_{1 \leq i \leq 4}$ Parse C_e as $s_1, s_2 := s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$ For $1 \leq i \leq 4$: $s_1 \leftarrow \text{HDPushForward}(s_1, s_{\text{HD}}^{\text{can}}(\delta_{e+1}^i))$ $s_2 \leftarrow \text{HDPushForward}(s_2, s_{\text{HD}}^{\text{can}}(\delta_{e+1}^i))$ $s_1 \leftarrow \text{ToCanonicalRepresentation}(s_1)$ $s_2 \leftarrow \text{ToCanonicalRepresentation}(s_2)$ return $C_{e+1} := (s_1, s_2)$	Parse C_e as $s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$ Parse k_e as I_e $s_1 \leftarrow \text{IdealPullBack}(I_e, s_{\text{HD}}^{\text{can}}(\varphi_1))$ $s_2 \leftarrow \text{IdealPullBack}(I_e, s_{\text{HD}}^{\text{can}}(\varphi_2))$ $x, y, z, t \leftarrow \text{HalfEndoToCoord}(s_1, s_2)$ $R' \ M' \leftarrow \pi^{-1}(z, t)$ return M'

Fig. 4. Our updatable encryption scheme DINE.

The above property is equivalent to requiring that $\Delta_{e+1} \circ \varphi_{k_e}$ and $\varphi_{k_{e+1}}$ have the same codomain, and that both send the cyclic subgroups of $E_0[N]$ to the same cyclic subgroups of $E_{e+1}[N]$. To translate this statement into the language of quaternions, we introduce some notations: let $\mathfrak{D} := \mathcal{O}_0$, $\mathfrak{D}' := \mathbb{Z} + N\mathcal{O}_0$, $\mathfrak{a} := I_e \cdot J_{e+1}$ and $\mathfrak{b} := I_{e+1}$.

By definition of the algorithm `SpecialKLPT` and the relation $\sim_{\mathfrak{D}'}$, we have that $K_{e+1} := \mathfrak{D}' \cap \mathfrak{b}$ is equivalent to $\mathfrak{D}' \cap \mathfrak{a}$. Then, [20, Lemma 1] implies the existence of $\alpha \in \mathfrak{D}' \cap \mathfrak{a}$ such that $\text{Nrd}(\alpha) = \text{Nrd}(\mathfrak{a}) \text{Nrd}(\mathfrak{b})$ and $\mathfrak{D}' \cap \mathfrak{b} = (\mathfrak{D}' \cap \mathfrak{a}) \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{a})$. Consequently, $\mathfrak{b} = \mathfrak{a} \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{a})$, and we obtain $\mathcal{O}_R(\mathfrak{a}) = \alpha / \text{Nrd}(\mathfrak{b}) \cdot \mathcal{O}_R(\mathfrak{b}) \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{a})$.

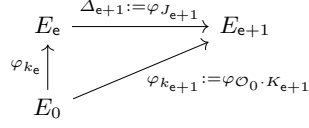


Fig. 5. The commutative diagram representing the key k_e together with the new key k_{e+1} and the update token Δ_{e+1} produced by **UpdateKey**. We keep the notations of Fig. 4.

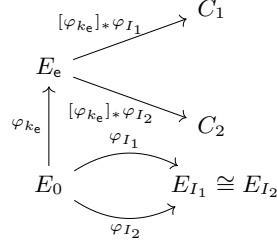


Fig. 6. Encryption of M into a ciphertext (C_1, C_2) under the key k_e . We keep the notations of Fig. 4.

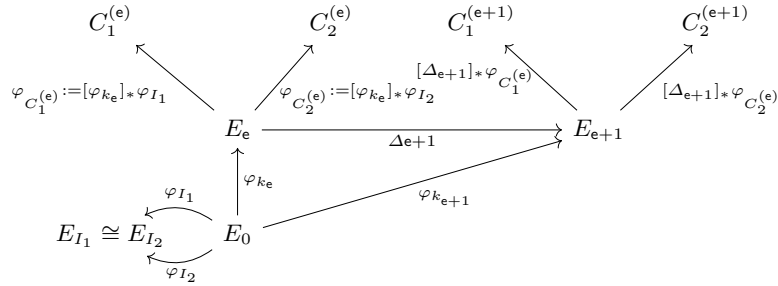


Fig. 7. Update of the ciphertext $(C_1^{(e)}, C_2^{(e)})$ under the key k_e to $(C_1^{(e+1)}, C_2^{(e+1)})$ under the key k_{e+1} using the token Δ_{e+1} . We keep the notations of Fig. 4.

Hence, in the quaternion world, the codomain condition is equivalent to $\mathcal{O}_R(\mathfrak{a})$ and $\mathcal{O}_R(\mathfrak{b})$ being isomorphic—an explicit isomorphism is provided in the previous paragraph. The condition on cyclic subgroups holds if the isogeny $\hat{\varphi}_{k_{e+1}} \circ \Delta_{e+1} \circ \varphi_{k_e}$ associated with the ideal $\mathfrak{a} \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{b}) \cdot \bar{\mathfrak{b}} \cdot \alpha / \text{Nrd}(\mathfrak{a})$ acts as scalar multiplication by an integer coprime to N on $E_0[N]$. From the relation $\mathfrak{b} = \mathfrak{a} \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{a})$ we deduce

$$\begin{aligned} \mathfrak{a} \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{b}) \cdot \bar{\mathfrak{b}} \cdot \alpha / \text{Nrd}(\mathfrak{a}) &= \mathfrak{a} \cdot \bar{\alpha} / \text{Nrd}(\mathfrak{b}) \cdot \alpha / \text{Nrd}(\mathfrak{a}) \cdot \bar{\mathfrak{a}} \cdot \alpha / \text{Nrd}(\mathfrak{a}) \\ &= \mathfrak{a} \cdot \bar{\mathfrak{a}} \cdot \alpha / \text{Nrd}(\mathfrak{a}) \\ &= \mathfrak{D} \text{Nrd}(\mathfrak{a}) \cdot \alpha / \text{Nrd}(\mathfrak{a}) \\ &= \mathfrak{D} \alpha, \text{ where } \alpha \in \mathfrak{D}' \cap \mathfrak{a} \subseteq \mathfrak{D}' \end{aligned}$$

Since $\alpha \in \mathfrak{D}'$ and $\text{Nrd}(\alpha) = (2^f - N)^{k+4} \text{Nrd}(\mathfrak{b})$ is coprime to N (noting that the reduced discriminant of $\mathfrak{D}'$ is pN^3 and, by definition of the relation $\sim_{\mathfrak{D}'}$, $\text{Nrd}(\mathfrak{b})$ is coprime to it), the cyclic subgroups condition is satisfied. Hence, the updates are indeed correct. $\square$

Proposition 2 (Correctness). *The DINE scheme of Fig. 4 is correct.*

Proof. Let $0 \leq e_1 \leq e_2 \leq n+1$ be two epochs, and consider a ciphertext $C^{(e_2)}$ obtained by successively updating an initial ciphertext $C^{(e_1)} := \text{Enc}(k_{e_1}, M)$

through the tokens Δ_{i+1} for $i \in [e_1, e_2 - 1]$, as in Definition 2. By definition, $C^{(e_1)} = ([\varphi_{I_{e_1}}]_* \varphi_{I_1}, [\varphi_{I_{e_1}}]_* \varphi_{I_2})$, where $I_1 := \mathcal{O}_0 \langle \theta_{N,M}, N \rangle$, $I_2 := \mathcal{O}_0 \langle \overline{\theta_{N,M}}, N \rangle$, and $\theta_{N,M} := x + yi + zj + tij$ satisfies $\text{Nrd}(\theta_{N,M}) = N^2$ with $z, t = \pi(R\|M)$.

By Proposition 1, updating $C^{(e_1)}$ through the sequence of tokens yields $C^{(e_2)} = ([\varphi_{I_{e_2}}]_* \varphi_{I_1}, [\varphi_{I_{e_2}}]_* \varphi_{I_2})$. Applying the `IdealPullBack` algorithm to I_{e_2} and the HD representations of $C_1^{(e_2)}$, $C_2^{(e_2)}$ recovers the HD representations of φ_{I_1} and φ_{I_2} . Then, `HalfEndoToCoord` applied to these representations returns the coordinates x, y, z, t of $\theta_{N,M}$, from which we recover the original message as $\pi^{-1}(z, t) = \pi^{-1}(\pi(R\|M)) = R\|M$.

The correctness of `IdealPullBack` and `HalfEndoToCoord` is established in Section 5.1 and Section 5.3, respectively. $\square$

4 Security

In this section, we prove the CPA security and ciphertext integrity of DINE, introduce the new hardness assumptions underlying its security, and deduce CCA security using the generic composition result of Boyd *et al.*

4.1 Security Assumptions

First and foremost, we need to identify the underlying hard problems of our scheme. To this end, we introduce several new problems and briefly discuss their presumed hardness. All of these problems involve computing pushforwards of N -isogenies through a secret $(2^f - N)$ -isogeny. Throughout, whenever we refer to an isogeny φ , we implicitly assume access to an efficient representation of φ , which is guaranteed by Kani's Lemma (see Section 2.2).

Definition 6 (Pseudorandom pushforward). *Let $\mathcal{O}_0$ be the special maximal order of $B_{p,\infty}$ with $p := c2^f - 1$, and let N be prime. Let $\mathcal{I}$ be a distribution over pairs of $\mathcal{O}_0$ -ideals of norm N . Define two distributions:*

- $\mathcal{D}_0$: *sample an $\mathcal{O}_0$ -ideal K of norm $2^f - N$; output $((\varphi_{I_1}, \varphi_{I_2}), ([\varphi_K]_* \varphi_{I_1}, [\varphi_K]_* \varphi_{I_2}))$, where $(I_1, I_2) \leftarrow \mathcal{I}$. Here K is fixed for all samples.*
- $\mathcal{D}_1$: *sample a maximal order $\mathcal{O}$ of $B_{p,\infty}$; output $((\varphi_{I_1}, \varphi_{I_2}), (\varphi_{J_1}, \varphi_{J_2}))$, where $(I_1, I_2) \leftarrow \mathcal{I}$ and (J_1, J_2) are uniformly random $\mathcal{O}$ -ideals of norm N .*

The advantage of an adversary $\mathcal{A}$ in breaking the pseudorandomness of the $(\mathcal{O}_0, N)$ -pushforward is $\text{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{A}) := |\Pr[\mathcal{A}^{\mathcal{D}_0} = 1] - \Pr[\mathcal{A}^{\mathcal{D}_1} = 1]|$.

Definition 7 (Unpredictable pushforward). *Using the notations of Definition 6, consider an adversary with oracle access to the distribution $\mathcal{D}_0$. Let K be the ideal sampled by $\mathcal{D}_0$ for computing pushforwards. The adversary's goal is to compute a new pair of N -isogenies (ψ_1, ψ_2) such that their pullbacks through φ_K , i.e., $([\varphi_K]_* \psi_1, [\varphi_K]_* \psi_2)$, form the decomposition of an endomorphism of degree*

N^2 . Let Q be the set of adversarial queries. The advantage of an adversary $\mathcal{A}$ in breaking the unpredictability of the $(\mathcal{O}_0, N)$ -pushforward is

$$\mathbf{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{UP}}(\mathcal{A}) := \Pr[(\psi_1, \psi_2) \leftarrow \mathcal{A}^{\mathcal{D}_0}; (\psi_1, \psi_2) \notin Q \wedge [\varphi_K]^*(\psi_1, \psi_2) \text{ forms a degree } N^2 \text{ endomorphism}]$$

Definition 8 (Decisional Isogeny Problem with Hidden Torsion Information). Let E be a supersingular curve over $\mathbb{F}_{p^2}$. Consider a secret bit $b \in \{0, 1\}$. The adversary $\mathcal{A}$ receives a supersingular curve E' defined as follows:

- If $b = 0$, E' is $(2^f - N)$ -isogenous to E via some isogeny $\psi : E \rightarrow E'$.
- If $b = 1$, E' is uniformly random.

Consider the following bijections σ_0, σ_1 between the sets of N -isogenies with domain E and E' :

- σ_0 maps an N -isogeny φ with domain E to its pushforward through ψ , i.e., $\sigma_0(\varphi) := [\psi]_*\varphi$.
- σ_1 is a uniformly random bijection between these two sets.

The adversary is given oracle access to σ_b and outputs a bit b' . The advantage of $\mathcal{A}$ in breaking the decisional $(2^f - N)$ -isogeny problem with hidden N -torsion information is

$$\mathbf{Adv}_{2^f - N, N}^{\text{DIPHTI}}(\mathcal{A}) := |\Pr[b' = b] - 1/2|$$

The security assumption behind the security of our scheme is the following:

Assumption 1. For $\lambda > 0$, in prime characteristic p of 2λ bits, if $2^f - N$ and N are primes of 2λ bits such that p has maximal order in $\mathbb{F}_N^\times$, and the distribution $\mathcal{I}$ has a support of size 2^λ then, for all probabilistic polynomial time adversaries $\mathcal{A}$, $\mathbf{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{A})$, $\mathbf{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{UP}}(\mathcal{A})$, and $\mathbf{Adv}_{2^f - N, N}^{\text{DIPHTI}}(\mathcal{A})$ are all $\text{negl}(\lambda)$.

We now justify its plausibility. First, without knowledge of the endomorphism ring of the domain, the computation of N and $(2^f - N)$ -isogenies has exponential complexity. The hardness of this prime-degree isogeny problem has recently been used in cryptographic constructions [4, 33]. While our new problems are not identical, this already suggests a significant obstacle to any efficient attack.

Our problems are more naturally viewed as variants of isogeny problems with torsion information (or level structure), first studied in the context of SIDH [24]. In these problems (at least the computational variants), the goal is to recover a secret isogeny of known degree given its action on some points of the domain. The security of SIDH relied on knowing the exact action of the secret isogeny on the full S -torsion for a smooth integer S , a case solvable in polynomial time using the same techniques we propose to use constructively in this work [41, 10, 35]. However, not all isogeny problems with torsion information are efficiently solvable. In particular, when the torsion information is not exact—for example, if the image points are scaled by a secret matrix—no efficient algorithm is known. Another crucial aspect to solve the problem efficiently is for the order of the torsion information to be smooth. A comprehensive survey on this topic is [19].

The reason the problems introduced above are related to isogeny problems with torsion information is that there exists a one-to-one correspondence between isogenies of degree N (coprime to p) and subgroups of order N of the domain. In principle, this correspondence implies that our problems are equivalent to isogeny problems with torsion information, since computing pushforwards is equivalent to evaluating on these subgroups. However, for appropriately chosen parameters, this equivalence may not be achievable in polynomial time.

As discussed in our technical overview, our goal is to operate precisely in the regime where it appears hard to relate the isogeny information provided to the adversary with explicit torsion information. To achieve this, N and p are chosen so that the N -torsion points of supersingular curves are defined only over an extension of $\mathbb{F}_p$ of exponential degree, making it infeasible to even write down a single point of order N . This is exactly why we describe the torsion information as “hidden” in Definition 8.

Thus, it appears difficult to use torsion points to attack our problems. Beyond that, and without directly recovering the secret $(2^f - N)$ -isogenies from their domain and codomain alone, there seems to be no obvious way to meaningfully exploit the additional information provided by the pushforwards. This is why we believe that Assumption 1 is plausible.

On the length of the tokens. In the overview of our scheme, we said that we need to take a minimum length of $k = 4$. This is because we want to approach a uniform distribution of keys in the class of $\mathbb{Z} + N\mathcal{O}_0$ -ideals (so that an adversary cannot distinguish between keys coming from `UpdateKey` and keys coming from `KeyGen`). The reduced discriminant of $\mathbb{Z} + N\mathcal{O}_0$ is $pN^3 \approx p^4$ and thus we need at least $k = 4$ to reach a distribution that is close to uniform heuristically. Note that we do not prove a small statistical distance to the uniform distribution, and thus implicitly rely on the heuristic assumption that the output of a random walk of length 4 in the isogeny graph of degree $2^f - N$ of supersingular elliptic curves enhanced with triples⁵ of disjoint torsion subgroups of order N is indistinguishable from a random such enhanced supersingular elliptic curve. To prove really something, one would probably need to go to $k = 8$, and generalize the usual proofs of mixing time for supersingular isogeny graphs to the case of supersingular elliptic curves enhanced with triples of disjoint subgroups of order N .

4.2 detIND-UE-CPA Security of DINE

In Theorem 3, we show that DINE achieves detIND-UE-CPA security in the ideal cipher model, assuming the hardness of the pseudorandom pushforward problem (Definition 6). The ideal cipher model, introduced by Shannon [42] and later shown equivalent to the random oracle model [14,23], provides access to a randomly chosen permutation from all possible key permutations of appropriate

⁵ the image of three disjoint subgroups is enough to determine the image of any subgroup of order N

length. Our scheme maps these permutation outputs to quaternions of a fixed norm, which allows the reduction to “program” this mapping.

Theorem 3 (DINE is detIND-UE-CPA). *Let DINE be the UE scheme described in Fig. 4 for the special maximal order $\mathcal{O}_0$ and integer N . For any ideal cipher model adversary $\mathcal{A}$, there exist reductions $\mathcal{B}$ and $\mathcal{B}'$ such that*

$$\mathbf{Adv}_{\text{DINE}}^{\text{detIND-UE-CPA}}(\mathcal{A}) \leq 2(n+1)^3(\mathbf{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{B}) + 2 \cdot \mathbf{Adv}_{2^f - N, N}^{\text{DIPHTI}}(\mathcal{B}'))$$

Proof overview. We provide a high-level overview of our proof here; the full details are given in Appendix F, along with a complete reduction shown in Fig. 11.

We follow the strategy of [8], using a hybrid argument over *insulated regions*. An insulated region is an epoch interval $[e_1, e_2]$ in which no keys are compromised, and neither the entering token Δ_{e_1} nor the exiting token Δ_{e_2+1} is revealed. The epochs e_1 and e_2 are referred to as the left and right *firewalls* of the region, respectively. Intuitively, a secure UE scheme should preserve ciphertext confidentiality within insulated regions. The detIND-UE-CPA notion formalizes this: the only epochs in which an adversary may request a challenge ciphertext without immediately triggering a trivial win must lie within such a region.

We use a hybrid argument over insulated regions. In the i -th hybrid, if the detIND-UE-CPA challenge occurs before (resp. after) the i -th insulated region, the challenger always returns an encryption of the plaintext (resp. an update of the ciphertext) supplied by the adversary. If the challenge occurs within the i -th region, a random bit decides whether an encryption or an update is returned.

We define a reduction $\mathcal{B}$ that receives a maximal order $\mathcal{O}_0$, an integer N , a distribution $\mathcal{I}$, and an oracle $\mathbf{BB}$ implementing either the distribution $\mathcal{D}_0$ or $\mathcal{D}_1$ of the PP experiment (cf. Definition 6). Concretely, $\mathbf{BB}$ returns either tuples of the form $((\varphi_{I_1}, \varphi_{I_2}), [\varphi_K]_*(\varphi_{I_1}, \varphi_{I_2}))$ or $((\varphi_{I_1}, \varphi_{I_2}), (\varphi_{J_1}, \varphi_{J_2}))$, where K is a fixed left $\mathcal{O}_0$ -ideal, $(I_1, I_2) \leftarrow \mathcal{I}$, and J_1, J_2 are random left $\mathcal{O}$ -ideals of norm N for a random maximal order $\mathcal{O}$. The reduction $\mathcal{B}$ uses samples from $\mathbf{BB}$ to simulate the detIND-UE-CPA experiment for DINE when $\mathbf{BB}$ implements $\mathcal{D}_0$ (and a random experiment otherwise). The core idea is to guess the left and right firewalls of the i -th insulated region and embed the isogeny φ_K into the epoch key of the left firewall. Specifically, $\varphi_{I_1}, \varphi_{I_2}$ correspond to the nonce-message pair (R, M) , and $[\varphi_K]_*(\varphi_{I_1}, \varphi_{I_2})$ serves as the ciphertext. Thus, an efficient adversary $\mathcal{A}$ against the detIND-UE-CPA security of DINE allows $\mathcal{B}$ —via the hybrid argument—to break the pseudorandomness of the $(\mathcal{O}_0, N)$ -pushforward.

However, this conclusion requires caution: it holds only if the reduction perfectly simulates the detIND-UE-CPA experiment for DINE when $\mathbf{BB}$ implements $\mathcal{D}_0$. In the case of DINE, we face two main challenges in establishing this claim.

First, in DINE, fresh ciphertexts are randomized via a nonce R , while updates are deterministic. The reduction must therefore ensure consistency of ciphertexts, *i.e.*, the nonce value R must remain consistent. To achieve this, we model the public permutation π in the ideal cipher model. Recall that π maps the concatenation of nonce and plaintext to a pair of integers in $[-N/\sqrt{2p}, N/\sqrt{2p}]$. In our proof, whenever an output of $\pi(\cdot)$ is needed, the reduction selects two

integers $z, t \in [-N/\sqrt{2p}, N/\sqrt{2p}]$ and programs $\pi(\cdot) = (z, t)$. In this setting, computing π^{-1} is simply a lookup to this mapping of the ideal cipher π .

When $\mathcal{B}$ receives an encryption query for a message M , it selects a random nonce R and queries BB to obtain $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2})$. It associates $\varphi_{I_1}, \varphi_{I_2}$ with the nonce-message pair (R, M) by computing the quaternion $\theta := x + yi + zj + tij \in B_{p,\infty}$ using the `HalfEndoToCoord` algorithm on $(\varphi_{I_1}, \varphi_{I_2})$, and programs $\pi(R\|M) \leftarrow (z, t)$. $\mathcal{B}$ then returns $[k]_*(\varphi_{I_1}, \varphi_{I_2})$ as ciphertext, where k is the current epoch key. Later, when updating the ciphertext to an epoch outside the i -th insulated region, $\mathcal{B}$ can compute the updated ciphertext as $[k']_*(\varphi_{I_1}, \varphi_{I_2})$ with the new epoch key k' , ensuring nonce consistency by correctness of DINE.

Moreover, when $\mathcal{B}$ must update this ciphertext to an epoch within the i -th insulated region, it uses $(\varphi_{J_1}, \varphi_{J_2})$ as ciphertext at the left firewall and updates it using simulated tokens for the remaining epochs of the region. If BB implements $\mathcal{D}_0$, we have $(\varphi_{J_1}, \varphi_{J_2}) := [k'']_*(\varphi_{I_1}, \varphi_{I_2})$ for some fixed key k'' , and by the correctness of DINE, nonce consistency is also preserved throughout the region.

Finally, our reduction embeds at the left firewall fwl_i a key generated by `KeyGen` rather than by applying `UpdateKey` to the previous epoch key. Likewise, since our reduction does not know the key at the right firewall fwr_i , it cannot derive k_{fwr_i+1} via `UpdateKey` and instead simulates it with `KeyGen`. A subtle point in our proof is that these keys are not a priori interchangeable: `KeyGen` keys are not guaranteed to admit an update token from the previous epoch, breaking the simulation in our hybrid argument. To address this, we introduce a new reduction to the DIPHTI problem of Definition 8, showing that even without admissible tokens the adversary's success probability does not change in a meaningful way.

4.3 detIND-UE-CCA Security of DINE

We prove that DINE achieves detIND-UE-CCA security under the unpredictable pushforward assumption (cf. Definition 7). Informally, it states that, given polynomially many tuples $(\varphi_{I_1}, \varphi_{I_2}, [\varphi_K]_*\varphi_{I_1}, [\varphi_K]_*\varphi_{I_2})$, no efficient adversary can generate a new pair (ψ_1, ψ_2) whose pullback $[\varphi_K]^*(\psi_1, \psi_2)$ decomposes into a degree N^2 endomorphism with non-negligible probability.

Theorem 4 (DINE is INT-CTXT^s). *Let DINE be the UE scheme described in Fig. 4 with the special maximal order $\mathcal{O}_0$ and $N \in \mathbb{N}$. For any ideal cipher model adversary $\mathcal{A}$, there exist reductions $\mathcal{B}$ and $\mathcal{B}'$ such that*

$$\text{Adv}_{\text{DINE}}^{\text{INT-CTXT}^s}(\mathcal{A}) \leq (n+1)(\text{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{UP}}(\mathcal{B}) + \text{Adv}_{2^f - N, N}^{\text{DIPHTI}}(\mathcal{B}'))$$

Corollary 1. *Combining the results of Theorems 1, 3 and 4, we conclude that DINE achieves detIND-UE-CCA security under Assumption 1.*

Proof. Let $\mathcal{A}$ be an adversary against INT-CTXT^s that outputs a valid forgery $\tilde{C}$ with probability $\Pr[E]$. We build a reduction $\mathcal{B}$, described in Fig. 12 of App. G, that wins the UP experiment with probability $\Pr[E]/(n+1)$.

The reduction $\mathcal{B}$ guesses the location of the left firewall fwl of the insulated region in which the $\mathcal{O}.\text{Try}$ query occurs, incurring a $(n+1)$ loss in advantage.

As noted by [36], guessing the right one is useless since the reduction can abort after the $\mathcal{O}.\text{Try}$ query. Then, $\mathcal{B}$ receives the UP parameters $(\mathcal{O}_0, N)$ together with oracle access to BB , which returns tuples $(\varphi_{I_1}, \varphi_{I_2}, [\varphi_K]_* \varphi_{I_1}, [\varphi_K]_* \varphi_{I_2})$ for a fixed left $\mathcal{O}_0$ -ideal K and $(I_1, I_2) \leftarrow \mathcal{I}$. $\mathcal{B}$ embeds φ_K into the epoch key $k_{\hat{\text{fwl}}}$ by using the two pushforwards provided by BB as ciphertexts in epoch $\hat{\text{fwl}}$.

Upon receiving a forgery $\tilde{C}$ in epoch $\tilde{e} \geq \hat{\text{fwl}}$, $\mathcal{B}$ “downgrades” $\tilde{C}$ to epoch $\hat{\text{fwl}}$ —where φ_K was embedded—by applying successive pullbacks through the simulated update tokens, denoted by $[\bigcirc_{e=\tilde{e}}^{\hat{\text{fwl}}+1} \Delta_e]_* \tilde{C}$. By correctness of DINE , $[\bigcirc_{e=\tilde{e}}^{\hat{\text{fwl}}+1} \Delta_e]_* \tilde{C}$ correctly decrypts whenever $\tilde{C}$ does. In that case, its pullback through φ_K is a degree N^2 endomorphism. Thus, $\mathcal{B}$ succeeds in the $\text{Exp}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{UP}}$ experiment with probability $\Pr[E]/(n+1)$.

Finally, $\mathcal{B}$ perfectly simulates the INT-CTXT^s experiment for DINE , by the same reasoning as in the proof of Th. 3, since all steps—including key embedding, key and token simulation, and nonce handling—are treated analogously. $\square$

5 Algorithmic Instantiation

This section provides detailed descriptions of the building blocks required by our UE construction, as listed in Section 3.1. We begin with algorithms that need no full description, as they can be derived from existing building blocks or explained informally.

- `EvalAnyIsoCanBasis` follows directly from `AnyIdealTolsogeny` [5, Algorithm 2].
- `PartialRepresentInteger` is obtained from `RepresentInteger` of [20] by taking the values z, t from input instead of generating them at random.
- `ToCanonicalRepresentation` computes a change-of-basis matrix from the domain basis of the input HD representation to the canonical basis, then applies it to the evaluation. The matrix is obtained by solving discrete logarithms in the 2^f -torsion group, which is efficient via the Pohlig–Hellman algorithm.

Moreover, a key tool for our construction is the arbitrary ideal-to-isogeny algorithm of [5], which runs in $O(\log(p \cdot \text{Nrd}(I)))$ and provides the following interface.

`AnyIdealTolsogeny` ([5, Algorithm 2]): given an ideal I of maximal orders in $B_{p, \infty}$, corresponding to an isogeny $\varphi_I : E \rightarrow E_I$, and a basis P, Q of $E[2^f]$, the algorithm outputs $\varphi_I(P), \varphi_I(Q)$.

Remark 2. While [5, Algorithm 2] assumes the domain of φ_I is a fixed starting curve E_0 , we note that the method extends naturally to any domain. One can apply the algorithm twice—once from E_0 to the domain curve, and once from E_0 to the codomain curve. Constructing ideals connecting $\mathcal{O}_0 \cong \text{End}(E_0)$ to any other maximal order is then straightforward.

The rest of this section is dedicated to the more involved algorithms `IdealPullBack`, `HDPushForward`, and `HalfEndoToCoord` which we construct explicitly.

5.1 Fixed Degree Pushforward and Arbitrary Degree Pullback

The function `HDPushForward` computes the pushforward of an isogeny of odd degree N through another isogeny of degree $2^f - N$. Thanks to the choice of degrees, this can be done efficiently via a direct application of Kani's Lemma. Consequently, the HD representation of the resulting pushforward isogeny can be obtained from a single 2^f -isogeny computation in dimension 2.

Algorithm 1 `HDPushForward`($s_{\text{HD}}(\varphi_1), s_{\text{HD}}(\varphi_2)$)

Require: $s_{\text{HD}}(\varphi_1)$ and $s_{\text{HD}}(\varphi_2)$, the canonical representations of two isogenies φ_1 and φ_2 of degrees N and $2^f - N$, respectively.

Ensure: $\perp$ or $s_{\text{HD}}([\varphi_2]_*\varphi_1)$

- 1: Parse $s_{\text{HD}}(\varphi_1), s_{\text{HD}}(\varphi_2)$ as E, P, Q, E_1, P_1, Q_1 and $E', P', Q', E_2, P_2, Q_2$
 - 2: If $E, P, Q \neq E', P', Q'$, return $\perp$
 - 3: Compute $F : E_1 \times E_2 \rightarrow E_3 \times E_4$ of kernel generated by (P_1, P_2) and (Q_1, Q_2)
 - 4: If the computation failed in any way or if $j(E_3) \neq j(E)$, return $\perp$
 - 5: Compute $*, P_4 \leftarrow F(0, P_2)$ and $*, Q_4 \leftarrow F(0, Q_2)$
 - 6: **return** $E_2, P_2, Q_2, E_4, P_4, Q_4$
-

Now, we provide a description of the `IdealPullBack` algorithm, which takes as input an isogeny ψ of degree N and domain E (given via its HD representation) and an ideal I of arbitrary norm⁶ with right order isomorphic to E , and outputs an HD representation of the pullback of ψ through φ_I . The main challenge for this algorithm is that, unlike in Section 5.1, the degree of φ_I is a power of $(2^f - N)$.

In this more general setting, performing the pullback with only a single 2^f -isogeny computation in dimension 2 via Kani's Lemma becomes highly impractical. Two issues arise. First, the norm $\text{Nrd}(I)$ of the ideal may be arbitrarily large, potentially exceeding 2^f . Second, dimension 2 provides very limited flexibility in choosing suitable degrees, even if $\text{Nrd}(I)$ is small enough. To make a single application of Kani's Lemma feasible, $\text{Nrd}(I)$ would have to belong to a very small list of values, which is extremely unlikely.

One way to overcome the second obstacle is to switch from dimension 2 to dimension 4, a now-classical trick in isogeny-based cryptography (see, *e.g.*, [39,16]). Of course dimension 4 is slower than dimension 2, but this slowdown is not prohibitive, as demonstrated by the implementation provided in [15,16]. However, our first issue remains: what if $\text{Nrd}(I)$ is too large? This question is relevant precisely because, in our application, it will indeed be the case.

Our solution is to replace I with another ideal J that is more convenient for computation while still producing the same output. For correctness, we require an ideal J such that $[\varphi_J]_*\psi = [\varphi_I]_*\psi$. As seen in Proposition 1, this holds when $I \sim_{\mathbb{Z} + N\mathcal{O}_L(I)} J$. Equivalently, we need to find $\beta \in I \cap (\mathbb{Z} + N\mathcal{O}_L(I))$ such that

⁶ in practice in our scheme the norm given will already be a power of $2^f - N$, but we can get a more generic version very easily thanks to the `SpecialKLPT` algorithm

Algorithm 2 $\text{IdealPullBack}(I, s_{\text{HD}}^{\text{can}}(\psi))$

Require: An ideal I of maximal orders in $B_{p,\infty}$, and $s_{\text{HD}}^{\text{can}}(\psi)$, the canonical representation of a N -isogeny ψ with domain's endomorphism ring isomorphic to $\mathcal{O}_R(I)$.

Ensure: $\perp$ or $s_{\text{HD}}^{\text{can}}([\varphi_I]^* \psi)$

- 1: **if** $n(I) = (2^f - N)^k$ **then**
- 2: $K \leftarrow I$
- 3: **else**
- 4: $K \leftarrow \text{SpecialKLPT}_{2^f - N}(I, N)$ of norm $(2^f - N)^e$
- 5: Parse $s_{\text{HD}}^{\text{can}}(\psi)$ as E, E', P', Q'
- 6: Compute P, Q (resp. P_0, Q_0) the canonical basis of E (resp. $E_0[2^f]$)
- 7: $R, S \leftarrow \text{AnyIdealTolsogeny}(K, P_0, Q_0)$
- 8: Find the matrix M such that $(R, S) = M(P, Q)$
- 9: $(R', S') \leftarrow M(P', Q')$
- 10: $s \leftarrow E, R, S, E', R', S'$
- 11: **for** $i = 1$ to e **do**
- 12: Parse s as E, P, Q, E', P', Q'
- 13: Factor K as $K_1 \cdot K_2$ where $\text{Nrd}(K_2) = 2^f - N$
- 14: $E_1, R_1, S_1 \leftarrow \text{AnyIdealTolsogeny}(K_1, P_0, Q_0)$
- 15: $s_1 \leftarrow E, P, Q, E_1, [(2^f - N)^i]R_1, [(2^f - N)^i]S_1$
- 16: $s \leftarrow \text{HDPushForward}(s, s_1)$
- 17: $K \leftarrow K_1$
- 18: **return** $\text{ToCanonicalRepresentation}(s)$

$M = \text{Nrd}(\beta) / \text{Nrd}(I)$ allows for efficient pullback computation. The ideal J is then computed as $J = I\bar{\beta} / \text{Nrd}(I)$. The remaining questions are: how do we choose M , and how do we find such a β ?

As explained, the main obstacle is the size of the degree. Thus, it is natural to first consider the expected size of the smallest suitable M . A simple covolume computation of the lattice $I \cap (\mathbb{Z} + N\mathcal{O}_L(I))$ shows that the expected size of the smallest element in this lattice is roughly $\text{Nrd}(I)\sqrt{pN^3}$, since the reduced discriminant of $\mathbb{Z} + N\mathcal{O}_L(I)$ equals pN^3 . Thus, the smallest possible M has size approximately $\sqrt{pN^3}$. Unfortunately, in our scheme we have $N \approx p$ yielding $M \approx p^2$ which is far larger than $2^f \approx p$. Hence, we cannot hope to find an ideal J with $\text{Nrd}(J) < 2^f$ that would allow computing the pullback with a single 2^f -isogeny computation in dimension 2 or 4.

Since we cannot hope to perform the pullback in a single step, we split the computation into smaller, manageable parts. In particular, if we can choose $M = (2^f - N)^e$ for some $e \in \mathbb{N}$, we will be able to compute the pullback via e sequential applications of Kani's Lemma. This approach appears to be the most practical. Finding a suitable β and ideal J can be done using a variant of the KLPT algorithm from [20, Appendix E], which we denote by $\text{SpecialKLPT}_{2^f - N}$.⁷

Next, we give a detailed description of the IdealPullBack algorithm, treating SpecialKLPT as a black box. In Section 5.2, we will describe SpecialKLPT in detail and analyze the expected size of the smallest output that can be computed

⁷ the version presented here is an improved version of the one outlined in [20]

efficiently. For simplicity, we assume the domain of φ_I is a public curve E_0 , which holds in the main variant of `IdealPullBack` used in DINE. The algorithm can be adapted to the generic setting required by the variant of our scheme in Appendix E by providing the domain of φ_I as input and replacing E_0 with it.

In `IdealPullBack`, for a pair of points P, Q , we define the action of a matrix $M \in \mathbb{Z}^{2 \times 2}$ as $R, S := M(P, Q)$ with $R = [M_{1,1}]P + [M_{1,2}]Q$ and $S = [M_{2,1}]P + [M_{2,2}]Q$. This notation will be reused later in Algorithm 5.

Proposition 3. (*Proof in Appendix H*) *Under plausible heuristic assumptions, the IdealPullBack algorithm is correct and runs in time $\text{poly}(\log(pN \text{Nrd}(I)))$.*

5.2 The Special KLPT Algorithm

In this section, we describe the algorithm `SpecialKLPT` _{M} , which takes as input an $\mathcal{O}_0$ -ideal I and an integer N , and outputs an element $\beta \in I \cap (\mathbb{Z} + N\mathcal{O}_0)$ of norm M^k with k as small as possible. Recall that $\mathcal{O}_0$ is a *special extremal order*, which enables the use of the most efficient KLPT variants (see [29, 20, 32]). Throughout this section, $\mathcal{O}_0$ is treated as an implicit parameter of the algorithms.

A heuristic algorithm for this task was given in [20, Appendix E], yielding $k \log M \approx 4 \log p + 5 \log N + 2 \log(\text{Nrd}(I))$. Without loss of generality, we may assume I has minimal norm in the equivalence class of $I \cap (\mathbb{Z} + N\mathcal{O}_0)$. By the gaussian heuristic, $\log(\text{Nrd}(I)) \approx 1/2 \log(p) + 3/2 \log(N)$, so the expected output size is about $5 \log(p) + 8 \log(N)$.

We now introduce a new heuristic algorithm achieving the same goal, but with expected output size $\approx 3 \log(p) + 7 \log N$. It has the same generic outline as all the KLPT algorithms:

1. Possibly replace I by a better suited ideal K with $I := K\delta$.
2. Generate $\gamma \in \mathcal{O}_0$ of norm $\text{Nrd}(K)M^{e_0}$ in a suitable suborder of $\mathcal{O}_0$.
3. Find μ_0 of arbitrary norm with $\gamma\mu_0 \in K$ and $\gamma\mu_0\delta$ in the target order.
4. Compute $\mu = \lambda\mu_0 + N \text{Nrd}(K)\mu_1$ of norm M^{e_1} .
5. Output $\beta = \gamma\mu\delta$.

In the algorithm from [20], one takes $K = I$ and $\gamma \in \mathbb{Z}\langle 1, Ni, Nj, Nk \rangle$. Our improvement is to instead choose K as the ideal of smallest prime norm in the equivalence class of $I \cap \mathbb{Z}\langle 1, i, Nj, Nk \rangle$ and $\gamma \in \mathbb{Z}\langle N, Ni, j, k \rangle$. These choices yield K and γ of smaller norms, thereby reducing the exponent k . Correctness hinges on the existence of a solution μ_0 in Step 3, which is established in Lemma 1.

The precise description will follow later in this section. We first introduce the necessary building blocks, some classical and others new. We begin with the well-known ones.

- **Cornacchia**: solves $x^2 + y^2 = m$, returning $\perp$ or integers x, y (see, e.g., [32, Algorithm 1]); for primes $m = 1 \pmod{4}$, the output is never $\perp$.
- **IdealModConstraint**: given an ideal K and γ , finds C, D with $\gamma j(C + iD) \in K$.
- **StrongApproximation _{ℓ}** : on input integers M, C, D , produces $\mu = \lambda j(C + iD) + M\mu_1$ with $\lambda \in \mathbb{Z}, \mu_1 \in \mathcal{O}_0$, ensuring $\text{Nrd}(\mu) = \ell^e$ for some e . The factorization of M is assumed known (see, e.g., [32, Algorithm 3]).

Special represent integer. To compute an element γ of prescribed norm in the order $\mathbb{Z}\langle N, Ni, j, k \rangle$, we adapt the classical `RepresentInteger` algorithm (see, *e.g.*, [32, Algorithm 2]), and denote this variant by `SpecialRepresentInteger`.

The main change is that, since we seek a solution of the form (Nx, Ny, z, t) , the coordinates z, t cannot be chosen at random as in `RepresentInteger`. Instead, M and $z^2 + t^2$ must satisfy a congruence modulo N^2 , so once $z^2 + t^2$ is chosen, the values of z, t are computed using `Cornacchia`'s algorithm.

The algorithm can also start by choosing x, y first, in which case the constraint is on $x^2 + y^2 \bmod p$. Since in our application $p \ll N^2$, selecting x, y before z, t will be more efficient.

Algorithm 3 `SpecialRepresentInteger`(M, N)

Require: Two integers N, M pairwise coprime and both coprime to p .

Ensure: $\perp$ or $\gamma \in \mathbb{Z}\langle N, Ni, j, k \rangle$ of norm M .

```

1: found  $\leftarrow \perp$ ;  $x, y, z, t \leftarrow 0, 0, 0, 0$ 
2:  $c \leftarrow M/N^2 \bmod p$ 
3:  $B \leftarrow M - cN^2$ 
4: while found =  $\perp$  and  $B > 0$  do
5:   if Cornacchia( $c$ )  $\neq \perp$  then
6:      $x, y \leftarrow \text{Cornacchia}(c)$ 
7:      $s \leftarrow B/p$ 
8:     if Cornacchia( $s$ )  $\neq \perp$  then
9:        $z, t \leftarrow \text{Cornacchia}(s)$ 
10:      found =  $\top$ 
11:     $B = B - pN^2$ 
12:     $c = c + p$ 
13: if found =  $\perp$  then return  $\perp$ 
14: return  $N(x + iy) + jz + kt$ 
```

Proposition 4. *Whenever `SpecialRepresentInteger` does not output $\perp$, its output is correct. Under plausible heuristic assumptions, for a security parameter λ , if $M > 5\lambda \log(p) \log(N)pN^2$, then the probability that `SpecialRepresentInteger` fails on input N, M is $\text{negl}(\lambda)$.*

Proof. Since `Cornacchia` never outputs $\perp$ when its input is a prime congruent to 1 mod 4, the probability that it fails on a random input smaller than some bound B is at most $C/(2 \log B)$ for some small constant C . Under the plausible assumption that the inputs provided to `Cornacchia` within `SpecialRepresentInteger` behave like random integers of the same size, we can lower-bound the success probability of the while loop on a value c by $1/(4 \log(c) \log((M - cN^2)/p))$. Since the first c is smaller than p , and the algorithm runs in time polynomial in $\log p$, we have $\log(c) < (1 + \varepsilon) \log p$ for some $\varepsilon > 0$. Therefore, the success probability is lower-bounded by $1/(2(1 + \varepsilon) \log(p) \log(M/p))$.

Thus, if $M > N^{2+\varepsilon}p$, the success probability is lower-bounded by $1/(2(1 + \varepsilon)(2 + \varepsilon) \log(p) \log(N))$. To achieve overwhelming success probability with re-

spect to a parameter λ , it suffices to try roughly $2\lambda(1+\varepsilon)(2+\varepsilon)\log(p)\log(N)$ candidates. Therefore, for $M > 5\lambda\log(p)\log(N)pN^2$, the failure probability of the `SpecialRepresentInteger` algorithm becomes $\text{negl}(\lambda)$, as claimed. $\square$

Projecting γ to $\mathbb{Z} + N\mathcal{O}_0$. We now focus on Step 3 of the `SpecialKLPT` algorithm. The key point is to ensure the existence of a solution μ_0 , which is the purpose of Lemma 1. Indeed, if it exists, then it is easy to find it with linear algebra. We denote the resulting algorithm by `NonGorensteinModConstraint`, which is closely related to `IdealModConstraint` and `EichlerModConstraint` (see, e.g., [32, Section 3.2.2]). A full description is omitted.

Lemma 1. *Let $\gamma \in \mathbb{Z}\langle N, Ni, j, k \rangle$ and $\delta \in \mathbb{Z}\langle 1, i, Nj, Nk \rangle$. Then there exists a nonzero element $\mu_0 \in \mathbb{Z}/N\mathbb{Z}\langle j, k \rangle$ such that $\gamma\mu_0\delta \in \mathbb{Z}\langle 1, Ni, Nj, Nk \rangle$.*

Proof. Write $\gamma = N(a+ib) + j(c+id)$, $\mu_0 = j(C+iD)$ and $\delta = x+iy + Nj(z+it)$. Modulo N , the product $\gamma\mu_0\delta$ simplifies to $-p(C+iD)(c-id)(x+iy)$. For this product to lie in $\mathbb{Z}\langle 1, Ni, Nj, Nk \rangle$, it suffices that the coefficient of i is 0 mod N . This coefficient is $C(-dz + cy) + D(cx + dy)$ and there are always non zero solutions C, D to this equation mod N . $\square$

The proof of Lemma 1 implicitly provides a simple method to compute μ_0 from N, γ , and δ . We denote this algorithm by `NonGorensteinModConstraint`(γ, δ, N) in the description of Algorithm 4.

Equivalent small ideal. The final building block for `SpecialKLPT` is an algorithm to replace the input ideal I with an equivalent ideal K of smaller norm. By Lem. 1, not all ideals suffice: K must be equivalent to I as $\mathbb{Z}\langle 1, i, Nj, Nk \rangle$ ideals. This ensures that the element δ satisfying $I = K\delta/\text{Nrd}(K)$ lies in $\mathbb{Z}\langle 1, i, Nj, Nk \rangle$.

We denote this algorithm by `EquivalentSpecialPrimalIdeal`. It is similar to the `EquivalentPrimalIdeal` algorithm ([32, Algorithm 6]). It consists in computing a reduced basis of the lattice $I \cap \mathbb{Z}\langle 1, i, Nj, Nk \rangle$ and enumerating short vectors until finding one $\hat{\delta}$ of norm $\text{Nrd}(I)N'$ for some prime N' . The ideal $K = I\hat{\delta}/\text{Nrd}(I)$ of norm N' is then returned. We omit a full description of the algorithm.

Detailed description of `SpecialKLPT`. We now give a full description of our heuristic algorithm. We also introduce a constant, `boundSRI`, which is used to select the exponent in the input to `SpecialRepresentInteger` to ensure overwhelming success probability. We do not provide a full proof of correctness for `SpecialKLPT`. It follows directly from Lemma 1 and the standard proof of the KLPT algorithm (see [32, Prop. 3.2.7]). The algorithm terminates with overwhelming probability provided `boundSRI` is chosen large enough so that Prop. 4 can be applied.

Regarding the output size, the usual estimate for `StrongApproximation`($x, \cdot, \cdot$) (see, e.g., [32, Lemma 3.1.6]) gives $\text{Nrd}(\mu) \approx p(N\text{Nrd}(K))^3$. We also have $\text{Nrd}(\gamma) \approx pN^2/\text{Nrd}(K)$, ignoring constants and log log factors from `boundSRI`. Hence, the final output has norm $\approx p^2N^5\text{Nrd}(K)^2$. By the gaussian heuristic, the norm of K is expected to be $\approx \sqrt{p}N$ (as the reduced discriminant of $\mathbb{Z}\langle N, Ni, j, k \rangle$ is $4pN^2$), so the generic expectation for the output of `SpecialKLPT` is around p^3N^7 .

Algorithm 4 SpecialKLPT_{M•}(I, N)

Require: $\mathcal{O}_0$ -ideal I and integer N such that $N, \text{Nrd}(I), p$ are all pairwise coprime.

Ensure: $\perp$ or $J \simeq_{\mathbb{Z}+N\mathcal{O}_0} I$ of norm M^e for some $e \in \mathbb{N}$.

```

1:  $K \leftarrow \text{EquivalentSpecialPrimaldeal}(I, N)$ 
2: Let  $\delta$  be the element such that  $I = K\delta/\text{Nrd}(K)$ 
3: Select  $M^{e_0} > pN^2\text{boundSRI}/\text{Nrd}(K)$ 
4:  $\mu \leftarrow \perp, \gamma \leftarrow 0$ 
5: while  $\mu = \perp$  and  $\gamma \neq \perp$  do
6:    $\gamma \leftarrow \text{SpecialRepresentInteger}(\text{Nrd}(K)M^{e_0})$ 
7:   if  $\gamma \neq \perp$  then
8:      $C_N, D_N \leftarrow \text{NonGorensteinModConstraint}(\gamma, \delta, N)$ 
9:      $C_K, D_K \leftarrow \text{IdealModConstraint}(K, \gamma)$ 
10:     $C \leftarrow \text{CRT}_{N, \text{Nrd}(K)}(C_N, C_K), D \leftarrow \text{CRT}_{N, \text{Nrd}(K)}(D_N, D_K)$ 
11:     $\mu \leftarrow \text{StrongApproximation}_{M^\bullet}(N \text{Nrd}(K), C, D)$ 
12: if  $\mu \neq \perp$  then
13:    $\beta \leftarrow \gamma\mu_0\delta/\text{Nrd}(K)$ 
14:   return  $J \leftarrow I\beta/\text{Nrd}(I)$ 
15: else return  $\perp$ 

```

5.3 Recovering the Coefficients of an Endomorphism from an HD Representation

In this section, we introduce the **HalfEndoToCoord** algorithm. Its goal is to recover the coordinates of an endomorphism $\theta = \hat{\varphi}_2 \circ \varphi_1 \in \text{End}(E_0)$ of norm N^2 in the basis $1, i, j, k$, given the HD representations of the isogenies φ_1, φ_2 .

The idea is to recover these coefficients by computing the four integers $\text{Trd}(\theta)$, $\text{Trd}(\theta i)$, $\text{Trd}(\theta j)$, and $\text{Trd}(\theta k)$ modulo 2^f . Indeed, if $\theta = x + iy + jz + kt$, then $\text{Trd}(\theta) = 2x$, $\text{Trd}(\theta i) = -2y$, $\text{Trd}(\theta j) = -2pz$, and $\text{Trd}(\theta k) = -2pt$. If the values of x, y, z, t are small enough, computing these traces modulo 2^f will yield their exact value over $\mathbb{Z}$, which suffices for our purpose. Modulo 2^f , the trace of an endomorphism θ (which can be evaluated on the 2^f -torsion) can be easily computed using the formula $\text{Trd}(\theta) = \theta + \bar{\theta}$.

Proposition 5. *Let $\theta = \hat{\varphi}_2 \circ \varphi_1$ be an endomorphism of norm N^2 corresponding to the quaternion $x+iy+jz+kt \in \mathbb{Z}^+\langle 1, i, j, k \rangle$, where $(x, y, z, t) \in [0, 2^{f-1}]$. Then **HalfEndoToCoord** successfully computes x, y, z, t on input $s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$.*

Proof. It is straightforward to verify that for $\theta = x+iy+jz+kt \in \mathbb{Z}\langle 1, i, j, k \rangle$, we have $\text{Trd}(\theta) = 2x$, $\text{Trd}(\theta i) = -2y$, $\text{Trd}(\theta j) = -2pz$ and $\text{Trd}(\theta k) = -2pt$. Thus, if the values $2x, 2y, 2z, 2t$ computed by **HalfEndoToCoord** correspond modulo 2^f to the traces of $\theta, \theta i, \theta j/p, \theta k/p$, then the output is correct. Indeed, since $x, y, z, t \in [0, 2^{f-1}]$, their doubles lie in $[0, 2^f]$. It remains to show that these computed values match the intended ones.

By the definition of the canonical HD representation, the points P_i, Q_i are equal to $\varphi_i(P_0, Q_0)$. If the matrix M is such that $M(P_2, Q_2) = \rho(P_1, Q_1)$, then $M\varphi_2(P_0, Q_0) = \rho \circ \varphi_1((P_0, Q_0))$ and thus $[N]M(P_0, Q_0) = \theta((P_0, Q_0))$. Consequently, the matrix representing the quaternion θi in the basis P_0, Q_0 is NMM_i .

Algorithm 5 HalfEndoToCoord($s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$)

Require: Canonical HD representations $s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$ of two N -isogenies φ_1, φ_2 with the same domain and isomorphic codomain.

Ensure: $\perp$ if the input is invalid; otherwise integers x, y, z, t such that the endomorphism $\theta = \hat{\varphi}_2 \circ \rho \circ \varphi_1$ corresponds to $x + iy + jz + kt$, where ρ is the isomorphism from the codomain of φ_1 to that of φ_2 .

- 1: Compute P_0, Q_0 , the canonical basis of $E_0[2^f]$
 - 2: **for** $\alpha \in \{\iota, \pi, \iota \circ \pi\}$, compute the matrices M_α s.t. $\alpha(P_0, Q_0) = M_\alpha(P_0, Q_0)$
 - 3: Parse $s_{\text{HD}}^{\text{can}}(\varphi_i)$ as E_0, E_i, P_i, Q_i for $i = 1, 2$
 - 4: **if** $j(E_1) \neq j(E_2)$, return $\perp$
 - 5: Compute the isomorphism $\rho : E_1 \rightarrow E_2$
 - 6: Find the matrix M s.t. $M(P_2, Q_2) = \rho(P_1, Q_1)M$
 - 7: Find the integer $0 \leq x \leq 2^{f-1}$ s.t. $2x = N(M_{0,0} + M_{1,1}) \bmod 2^f$
 - 8: Find the integer $0 \leq y \leq 2^{f-1}$ s.t. $2y = -N(M_{0,0}M_{\iota,0,0} + M_{0,1}M_{\iota,1,0} + M_{0,1}M_{\iota,1,0} + M_{1,1}M_{\iota,1,1}) \bmod 2^f$
 - 9: $\kappa \leftarrow p^{-1} \bmod 2^f$
 - 10: Find the integer $0 \leq z \leq 2^{f-1}$ s.t. $2z = -\kappa N(M_{0,0}M_{\pi,0,0} + M_{0,1}M_{\pi,1,0} + M_{0,1}M_{\pi,1,0} + M_{1,1}M_{\pi,1,1}) \bmod 2^f$
 - 11: Find the integer $0 \leq t \leq 2^f$ s.t. $2t = -\kappa N(M_{0,0}M_{\iota \circ \pi,0,0} + M_{0,1}M_{\iota \circ \pi,1,0} + M_{0,1}M_{\iota \circ \pi,1,0} + M_{1,1}M_{\iota \circ \pi,1,1}) \bmod 2^f$
 - 12: **return** x, y, z, t
-

whose trace is $N(M_{0,0}M_{\iota,0,0} + M_{0,1}M_{\iota,1,0} + M_{0,1}M_{\iota,1,0} + M_{1,1}M_{\iota,1,1})$. Applying the same reasoning to j, k yields the desired result. $\square$

6 Instantiation of the Scheme

In this section, we describe a concrete instantiation of DINE and propose parameter choices targeting NIST security level 1. We also provide a C implementation built on top of the latest implementation of SQIsign (see <https://sqisign.org>).

6.1 Parameters

We use the same prime characteristic $p = 5 \cdot 2^{248} - 1$ as [5] to enable efficient 2^f -isogeny computation in dimension 2. We then choose $N = 2^{247} - 28899$. This is the largest prime smaller than 2^{247} satisfying all the necessary constraints. We looked for primes smaller than 2^{247} so that $2^{248} - N$ is not too small.

One can verify that N is prime and that p has maximal order $N - 1$ in $\mathbb{F}_N^\times$. For good measure, we also chose N such that $2^{248} - N$ is prime, making the computation of $(2^f - N)$ -isogenies as hard as possible (hoping to make problem DIPHTI as hard as possible). While a non-prime $2^{248} - N$ would likely not compromise security, imposing this condition is easy given the freedom we have in our choice of N and positions us in the hardest plausible setting.

With these choices, we can justify the sizes of the various elements in our scheme presented at the beginning of the paper. We can encrypt messages of

size $m(\lambda) = 8 \cdot 28 = 224$ bits (28 bytes), which is the largest multiple of 8 that allows to choose $n(\lambda)$ large enough to provide sufficient randomization for `PartialRepresentInteger` to succeed with overwhelming probability. Similarly to `SpecialRepresentInteger`, the failure probability of `PartialRepresentInteger` is in $O(\log p)$, so $n(\lambda)$ must be large enough to ensure that encryption failures are negligible. With $m(\lambda) = 28 \times 8$, taking $n(\lambda) = 15$ appears adequate.

Ciphertexts consist of two curves (the codomains of the two ciphertext isogenies) and two bases (the images through the ciphertext isogenies of the deterministic basis of the domain). Tokens consist of four times: two curves (the domain and codomain of the token isogeny) and one basis (the image of the deterministic basis of the domain through the token isogeny). In the Montgomery model, each elliptic curve can be represented by a single element over $\mathbb{F}_{p^2}$ (for instance, the coefficient A). Using x -coordinates, a basis can be represented by 3 elements over $\mathbb{F}_{p^2}$. For a 2^f -basis, this representation can be compressed to 3 elements of f bits, roughly halving its size. This compression is a common technique [5,17] and relies on pairings to convert between the full and compressed representations.

In our setting, encoding elements of $\mathbb{F}_p$ (and elements of f bits) in 32 bytes gives a total compressed size of 320B for ciphertexts and 896B for tokens.

6.2 Implementation Details

To improve the efficiency of our scheme, certain values can be precomputed during key generation or key update and stored as part of the secret key. In particular, the chain of 2^f -isogenies used in `IdealPullBack` during decryption is identical for ciphertexts encrypted with the same key. Thus, this chain can be computed once and stored. Similarly, the evaluation of the secret key ideal I_e on the canonical basis of E_0 can be precomputed to avoid calling `AnyIdealTolsogeny` on K for computing the initial R, S in `IdealPullBack`.

These precomputations eliminate all calls to `AnyIdealTolsogeny` in `IdealPullBack` during decryption, substantially speeding up the process, as shown by the timings reported in Table 2.

6.3 Results

Experimental results from our C implementation are reported in Table 2. As intended, `Upd` is significantly faster than the other algorithms, while encryption and decryption remain reasonably efficient. The running time of `KeyGen` is much longer, and we observed that it can vary a lot in practice. Most of this time is spent in `SpecialKLPT`, whose running time can vary a lot due to occasional failures. While it may be possible to optimize `SpecialKLPT`, we did not explore this, as key generation times are not critical in our setting.

Note that a very recent article [7] significantly improves the efficiency of `AnyIdealTolsogeny`—reporting roughly $2\times$ speedup at NIST level 1. This improvement could be directly applied to DINE, reducing the running time of the

isogeny computations in **KeyGen** (*i.e.*, everything except **SpecialKLPT**) and also speeding up **Enc** by a good factor, while leaving the other operations unaffected.

References

1. Marius A Aardal, Andrea Basso, Luca De Feo, Sikhar Patranabis, and Benjamin Wesolowski, *A complete security proof of sqsign*, Annual International Cryptology Conference, Springer, 2025, pp. 190–222.
2. Navid Alapati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis, *Cryptographic Group Actions and Applications*, Advances in Cryptology – ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II (Berlin, Heidelberg), Springer-Verlag, 2020, p. 411–439.
3. Karim Bagheri, Daniele Cozzo, and Robi Pedersen, *An Isogeny-Based ID Protocol Using Structured Public Keys*, Cryptography and Coding (Cham) (Maura B. Paterson, ed.), Springer International Publishing, 2021, pp. 179–197.
4. Andrea Basso, Giacomo Borin, Wouter Castryck, Maria Corte-Real Santos, Riccardo Invernizzi, Antonin Leroux, Luciano Maino, Frederik Vercauteren, and Benjamin Wesolowski, *Prism: Simple and compact identification and signatures from large prime degree isogenies*, IACR International Conference on Public-Key Cryptography, Springer, 2025, pp. 300–332.
5. Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski, *SQIsign2D–West: The Fast, the Small, and the Safer*, Advances in Cryptology – ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings, Part III (Berlin, Heidelberg), Springer-Verlag, 2024, p. 339–370.
6. Dan Boneh, Kevin Lewi, Hart Montgomery, and Ananth Raghunathan, *Key Homomorphic PRFs and Their Applications*, Advances in Cryptology – CRYPTO 2013 (Berlin, Heidelberg) (Ran Canetti and Juan A. Garay, eds.), Springer Berlin Heidelberg, 2013, pp. 410–428.
7. Giacomo Borin, Maria Corte-Real Santos, Jonathan Komada Eriksen, Riccardo Invernizzi, Marzio Mula, Sina Schaeffler, and Frederik Vercauteren, *Qlapoti: Simple and efficient translation of quaternion ideals to isogenies*, Cryptology ePrint Archive (2025).
8. Colin Boyd, Gareth T. Davies, Kristian Gjøsteen, and Yao Jiang, *Fast and Secure Updatable Encryption*, Advances in Cryptology – CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part I, Springer-Verlag, 2020, p. 464–493.
9. Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes, *CSIDH: an efficient post-quantum commutative group action*, International Conference on the Theory and Application of Cryptology and Information Security, Springer, 2018, pp. 395–427.
10. Wouter Castryck, Jana Sotáková, and Frederik Vercauteren, *Breaking the decisional diffie-hellman problem for class group actions using genus theory*, Annual International Cryptology Conference, Springer, 2020, pp. 92–120.
11. Mingjie Chen, Muhammad Imran, Gábor Ivanyos, Péter Kutas, Antonin Leroux, and Christophe Petit, *Hidden stabilizers, the isogeny to endomorphism ring problem and the cryptanalysis of psidh*, International Conference on the Theory and Application of Cryptology and Information Security, Springer, 2023, pp. 99–130.

12. Mingjie Chen, Antonin Leroux, and Lorenz Panny, *SCALLOP-HD: Group Action from 2-Dimensional Isogenies*, Public-Key Cryptography – PKC 2024 (Cham) (Qiang Tang and Vanessa Teague, eds.), Springer Nature Switzerland, 2024, pp. 190–216.
13. Mingjie Chen and Christophe Petit, *Computing the endomorphism ring of a supersingular elliptic curve from a full rank suborder*, Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2025, pp. 446–474.
14. Jean-Sébastien Coron, Jacques Patarin, and Yannick Seurin, *The Random Oracle Model and the Ideal Cipher Model Are Equivalent*, Advances in Cryptology – CRYPTO 2008 (Berlin, Heidelberg) (David Wagner, ed.), Springer Berlin Heidelberg, 2008, pp. 1–20.
15. Pierrick Dartois, *Fast computation of 2-isogenies in dimension 4 and cryptographic applications*, Journal of Algebra **683** (2025), 449–514.
16. Pierrick Dartois, Jonathan Komada Eriksen, Tako Boris Fouotsa, Arthur Herlédan Le Merdy, Riccardo Invernizzi, Damien Robert, Ryan Rueger, Frederik Vercauteren, and Benjamin Wesolowski, *Pegasis: Practical effective class group action using 4-dimensional isogenies*, Annual International Cryptology Conference, Springer, 2025, pp. 67–99.
17. Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski, *Sqisignhd: new dimensions in cryptography*, Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2024, pp. 3–32.
18. Pierrick Dartois, Luciano Maino, Giacomo Pope, and Damien Robert, *An Algorithmic Approach to (2,2)-Isogenies in the Theta Model and Applications to Isogeny-Based Cryptography*, Advances in Cryptology – ASIACRYPT 2024 (Singapore) (Kai-Min Chung and Yu Sasaki, eds.), Springer Nature Singapore, 2025, pp. 304–338.
19. Luca De Feo, Tako Boris Fouotsa, and Lorenz Panny, *Isogeny problems with level structure*, Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2024, pp. 181–204.
20. Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski, *SQISign: compact post-quantum signatures from quaternions and isogenies*, Advances in Cryptology–ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part I 26, Springer, 2020, pp. 64–93.
21. Julien Duman, Dominik Hartmann, Eike Kiltz, Sabrina Kunzweiler, Jonas Lehmann, and Doreen Riepel, *Generic Models for Group Actions*, Public-Key Cryptography – PKC 2023: 26th IACR International Conference on Practice and Theory of Public-Key Cryptography, Atlanta, GA, USA, May 7–10, 2023, Proceedings, Part I (Berlin, Heidelberg), Springer-Verlag, 2023, p. 406–435.
22. Joseph H. Silverman, *The arithmetic of elliptic curves*, vol. 106, 01 2009.
23. Thomas Holenstein, Robin Künzler, and Stefano Tessaro, *The Equivalence of the Random Oracle Model and the Ideal Cipher Model, Revisited*, Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing (New York, NY, USA), STOC '11, Association for Computing Machinery, 2011, p. 89–98.
24. David Jao and Luca De Feo, *Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies*, Post-Quantum Cryptography (Berlin, Heidelberg) (Bo-Yin Yang, ed.), Springer Berlin Heidelberg, 2011, pp. 19–34.

25. Yao Jiang, *The Direction of Updatable Encryption Does Not Matter Much*, Advances in Cryptology – ASIACRYPT 2020 (Cham) (Shiho Moriai and Huaxiong Wang, eds.), Springer International Publishing, 2020, pp. 529–558.
26. Yao Jiang Galteland and Jiaxin Pan, *Backward-Leak Uni-Directional Updatable Encryption from (Homomorphic) Public Key Encryption*, Public-Key Cryptography – PKC 2023: 26th IACR International Conference on Practice and Theory of Public-Key Cryptography, Atlanta, GA, USA, May 7–10, 2023, Proceedings, Part II (Berlin, Heidelberg), Springer-Verlag, 2023, p. 399–428.
27. Ernst Kani, *The number of curves of genus two with elliptic differentials*, Journal für die reine und angewandte Mathematik **485** (1997), 93–122.
28. Michael Klooß, Anja Lehmann, and Andy Rupp, *(R)CCA Secure Updatable Encryption with Integrity Protection*, Advances in Cryptology – EUROCRYPT 2019 (Cham) (Y. Ishai and V. Rijmen, eds.), Springer International Publishing, 2019, pp. 68–99.
29. David Kohel, Kristin E. Lauter, Christophe Petit, and Jean-Pierre Tignol, *On the quaternion ℓ -isogeny path problem*, IACR Cryptology ePrint Archive **2014** (2014), 505.
30. Anja Lehmann and Björn Tackmann, *Updatable Encryption with Post-Compromise Security*, Advances in Cryptology – EUROCRYPT 2018 (Cham) (J. B. Nielsen and V. Rijmen, eds.), Springer International Publishing, 2018, pp. 685–716.
31. Antonin Leroux, *A new isogeny representation and applications to cryptography*, International Conference on the Theory and Application of Cryptology and Information Security, Springer, 2022, pp. 3–35.
32. ———, *Quaternion algebra and isogeny-based cryptography*, Ph.D. thesis, PhD thesis, Ecole doctorale de l’Institut Polytechnique de Paris, 2022.
33. ———, *Verifiable random function from the deuring correspondence and higher dimensional isogenies*, Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2025, pp. 167–194.
34. Antonin Leroux and Maxime Roméas, *Updatable Encryption from Group Actions*, Post-Quantum Cryptography: 15th International Workshop, PQCrypto 2024, Oxford, UK, June 12–14, 2024, Proceedings, Part II (Berlin, Heidelberg), Springer-Verlag, 2024, p. 20–53.
35. Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski, *A direct key recovery attack on sidh*, EUROCRYPT, 2023.
36. Jonas Meers and Doreen Riepel, *CCA Secure Updatable Encryption from Non-mappable Group Actions*, Post-Quantum Cryptography: 15th International Workshop, PQCrypto 2024, Oxford, UK, June 12–14, 2024, Proceedings, Part I (Berlin, Heidelberg), Springer-Verlag, 2024, p. 137–169.
37. T. Moriya, H. Onuki, and T. Takagi, *SiGamal: a supersingular isogeny-based PKE and its application to a PRF*, International Conference on the Theory and Application of Cryptology and Information Security, Springer, 2020, pp. 551–580.
38. Ryo Nishimaki, *The Direction of Updatable Encryption Does Matter*, Public-Key Cryptography – PKC 2022: 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, Virtual Event, March 8–11, 2022, Proceedings, Part II (Berlin, Heidelberg), Springer-Verlag, 2022, p. 194–224.
39. Aurel Page and Damien Robert, *Introducing Clapoti(s): Evaluating the isogeny class group action in polynomial time*, Cryptology ePrint Archive, Paper 2023/1766, 2023.
40. Arnold Pizer, *An algorithm for computing modular forms on $\Gamma_0(N)$* , Journal of Algebra **64** (1980), no. 2, 340–390.

41. Damien Robert, *Breaking sidh in polynomial time*, Advances in Cryptology – EUROCRYPT 2023 (Cham) (Carmit Hazay and Martijn Stam, eds.), Springer Nature Switzerland, 2023, pp. 472–503.
42. C. E. Shannon, *Communication theory of secrecy systems*, The Bell System Technical Journal **28** (1949), no. 4, 656–715.
43. Daniel Slamanig and Christoph Striecks, *Revisiting Updatable Encryption: Controlled Forward Security, Constructions and a Puncturable Perspective*, Theory of Cryptography: 21st International Conference, TCC 2023, Taipei, Taiwan, November 29 – December 2, 2023, Proceedings, Part II (Berlin, Heidelberg), Springer-Verlag, 2023, p. 220–250.
44. J. Vélú, *Isogénies entre courbes elliptiques*, Comptes-Rendus de l’Académie des Sciences, Série I **273** (1971), 238–241.
45. John Voight, *Quaternion algebras*, Springer International Publishing, Cham, 2021.
46. Benjamin Wesolowski, *The supersingular isogeny path and endomorphism ring problems are equivalent*, 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS), 2022, pp. 1100–1111.

A Computing Leakage Sets

Following [30], extended epoch leakage sets $\mathcal{C}^*$, $\mathcal{K}^*$ and $\mathcal{T}^*$ are computed as follows:

$$\begin{aligned}
 \mathcal{K}^* &\leftarrow \{e \in \{0, \dots, n\} \mid \text{CorrK}(e) = \text{true}\} \\
 \text{true} &\leftarrow \text{CorrK}(e) \Leftrightarrow (e \in \mathcal{K}) \vee (\text{CorrK}(e-1) \wedge e \in \mathcal{T}) \vee (\text{CorrK}(e+1) \wedge e+1 \in \mathcal{T}) \\
 \mathcal{T}^* &\leftarrow \{e \in \{0, \dots, n\} \mid (e \in \mathcal{T}) \vee (e \in \mathcal{K}^* \wedge e-1 \in \mathcal{K}^*)\} \\
 \mathcal{C}^* &\leftarrow \{e \in \{0, \dots, n\} \mid \text{ChallEq}(e) = \text{true}\} \\
 \text{true} &\leftarrow \text{ChallEq}(e) \Leftrightarrow (e = \tilde{e}) \vee (e \in \mathcal{C}) \vee (\text{ChallEq}(e-1) \wedge e \in \mathcal{T}^*) \vee \\
 &\quad (\text{ChallEq}(e+1) \wedge e+1 \in \mathcal{T}^*)
 \end{aligned}$$

Likewise, we extend $\mathcal{I}$ into $\mathcal{I}^*$:

$$\begin{aligned}
 \mathcal{I}^* &\leftarrow \{e \in \{0, \dots, n\} \mid \text{ChallInpEq}(e) = \text{true}\} \\
 \text{true} &\leftarrow \text{ChallInpEq}(e) \Leftrightarrow (e \in \mathcal{I}) \vee (\text{ChallInpEq}(e-1) \wedge e \in \mathcal{T}^*) \vee \\
 &\quad (\text{ChallInpEq}(e+1) \wedge e+1 \in \mathcal{T}^*)
 \end{aligned}$$

Moreover, Fig. 8 shows how to extend $\mathcal{L}$ into $\mathcal{L}^*$.

B The Check Algorithm

In our proofs, reductions play hybrid games and guess the location of the i -th insulated region. If the adversary sends a corrupt query inside this insulated region, the guess is wrong and reductions have to abort. We use the algorithm Check of [8], described in Fig. 9, to check if this event happens.

Update $\mathcal{L}^*$
if $\mathcal{O}.\text{Enc}$ or $\mathcal{O}.\text{Upd}$ happens $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{(\cdot, C, \cdot)\}$ if $\mathcal{O}.\text{Corr}(\text{token}, \cdot)$ happens for $i \in \mathcal{T}^*$: for $(j, C_{i-1}, i-1) \in \mathcal{L}^*$: $C_i \leftarrow \text{UE.Upd}(\Delta_i, C_{i-1})$ $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{(j, C_i, i)\}$

Fig. 8. Update procedure of [8] for list $\mathcal{L}^*$.

Check(inp, $\hat{e}$; e; fwl, fwr)
if $\hat{e} > e$ then return $\perp$ if inp = key and $\hat{e} \in [\text{fwl}, \text{fwr}]$ then return ABORT if inp = token and $\hat{e} \in [\text{fwl}, \text{fwr} + 1]$ then return ABORT

Fig. 9. Algorithm Check of [8] used in our proofs. $\hat{e}$ is the epoch in the adversary's request and e is the current epoch.

C Background on Isogenies

Let K be a finite field of characteristic $p > 0$ and E_1, E_2 elliptic curves over K . An *isogeny* $\varphi : E_1 \rightarrow E_2$ is a non-constant morphism sending the identity of E_1 to that of E_2 . Its *degree* is defined as the degree of the corresponding rational map. If $\deg(\varphi) = d$ is coprime to p , then φ is *separable*, with kernel a subgroup of order d . Such an isogeny is completely determined (up to isomorphism of the target curve) by its kernel, yielding a one-to-one correspondence between separable isogenies and finite subgroups of $E_1(\overline{K})$. Given a subgroup G , the isogeny with kernel G is computed using Vélu's formula [44], and written $\varphi : E \rightarrow E/G$. The degree of $\varphi \circ \psi$ is equal to $\deg(\varphi) \deg(\psi)$. Each isogeny $\varphi : E_1 \rightarrow E_2$ admits a unique *dual isogeny* $\hat{\varphi} : E_2 \rightarrow E_1$ such that $\varphi \circ \hat{\varphi} = [\deg(\varphi)]$, the multiplication-by- $\deg(\varphi)$ map on E_2 and $\hat{\varphi} \circ \varphi = [\deg(\varphi)]$ on E_1 . An *endomorphism* of E is an isogeny $\theta : E \rightarrow E$ or the constant map [0]. The set of endomorphisms $\text{End}(E)$ forms a ring under addition and composition. A curve E over K is called *supersingular* if $\text{End}(E)$ is a maximal order inside the quaternion algebra $B_{p,\infty}$ ramified at p and ∞ .

Finally, the Frobenius morphism $\pi : (x, y) \rightarrow (x^p, y^p)$ sends a curve $E : y^2 = x^3 + ax + b$ to $E^{(p)} : y^2 = x^3 + a^p x + b^p$. Moreover, π is the only isogeny of degree p between any two supersingular curves.

D Background on the Deuring Correspondence

Let $a, b \in \mathbb{Q}^\times$, and let $B(a, b)$ denote the quaternion algebra over $\mathbb{Q}$ with basis $1, i, j, ij$ and relations $i^2 = a, j^2 = b$ and $ij = -ji$. There is a *canonical involution* on B mapping $\alpha := x + yi + zj + tij$ to $\bar{\alpha} := x - yi - zj - tij$. The *reduced trace* and *reduced norm* of α are defined as $\text{Trd}(\alpha) := \alpha + \bar{\alpha} = 2x$ and $\text{Nrd}(\alpha) := \alpha\bar{\alpha} = x^2 - ay^2 - bz^2 + abt^2$.

A lattice I in B is a $\mathbb{Z}$ -module of the form $I := \mathbb{Z}b_1 + \mathbb{Z}b_2 + \mathbb{Z}b_3 + \mathbb{Z}b_4$, where the b_i 's form a basis for the $\mathbb{Q}$ -vector space B . The *reduced norm* of I is defined as $\text{Nrd}(I) := \gcd(\{\text{Nrd}(\alpha) \mid \alpha \in I\})$. An *order* $\mathcal{O}$ in B is a lattice that is also a subring of B . An order is *maximal* if it is not contained in any other order. For a lattice I , the *left* and *right orders* of I are defined as:

$$\mathcal{O}_L(I) := \{\alpha \in B \mid \alpha I \subseteq I\} \quad \text{and} \quad \mathcal{O}_R(I) := \{\alpha \in B \mid I\alpha \subseteq I\}$$

If $\mathcal{O}$ is a maximal order and I a left $\mathcal{O}$ -ideal, then $\mathcal{O}_R(I)$ is itself a maximal order. Given two maximal orders $\mathcal{O}$ and $\mathcal{O}'$, there exists a lattice I , called a *connecting ideal*, such that $\mathcal{O}_L(I) = \mathcal{O}$ and $\mathcal{O}_R(I) = \mathcal{O}'$.

Two $\mathcal{O}$ -ideals I and J are said to be *equivalent* if there exists $\alpha \in B^\times$ such that $I = J\alpha$, which we denote $I \sim J$. The set of equivalence classes under $\sim$ is the *left-ideal class set* of $\mathcal{O}$, denoted $\text{Cls}(\mathcal{O})$. Let $\mathfrak{D} \subset \mathcal{O}$ be a quaternion suborder. If $I \sim J$ are two $\mathcal{O}$ -ideals of norms coprime to $\text{disc } \mathfrak{D}$, we write $I \sim_{\mathfrak{D}} J$ if the element $\alpha \in B^\times$ such that $I\bar{J} = \mathcal{O}_L(J)\alpha$ belongs to $\mathfrak{D}$.

Let p be a prime congruent to 3 mod 4. Then $B_{p,\infty} = B(-1, -p)$ [40], where $B_{p,\infty}$ denotes the unique quaternion algebra ramified exactly at p and ∞ . In this case, $B_{p,\infty}$ contains the maximal order $\mathcal{O}_0 := \langle 1, i, \frac{1+i}{2}, i\frac{1+i}{2} \rangle$. Throughout this paper, p is always a prime congruent to 3 mod 4, and $\mathcal{O}_0$ always denotes this specific maximal order.

The Deuring correspondence associates each maximal order $\mathcal{O}$ in $B_{p,\infty}$ with a supersingular elliptic curve E whose endomorphism ring $\text{End}(E)$ is isomorphic to $\mathcal{O}$. That is, there is a bijection between the sets

$$\left\{ \begin{array}{l} \text{Isomorphism classes of} \\ \text{maximal orders in } B_{p,\infty} \end{array} \right\} \leftrightarrow \left\{ \begin{array}{l} \text{Isomorphism classes of} \\ \text{supersingular elliptic curves} \end{array} \right\} / \text{Gal}(\mathbb{F}_p^2 / \mathbb{F}_p)$$

In our setting, the supersingular curve E_0 defined over $\mathbb{F}_p$ by $y^2 = x^3 - x$ corresponds to the maximal order $\mathcal{O}_0$. Let π denote the Frobenius endomorphism $(x, y) \mapsto (x^p, y^p)$ and ι the endomorphism $(x, y) \mapsto (-x, \alpha y)$, where $\alpha \in \mathbb{F}_{p^2}$ satisfies $\alpha^2 = -1$. Then $\text{End}(E_0) = \mathbb{Z} + \mathbb{Z}\iota + \mathbb{Z}\frac{1+\pi}{2} + \mathbb{Z}\iota\frac{1+\pi}{2}$. Since $\iota^2 = [-1]$ and $\pi^2 = [-p]$, we have $\text{End}(E_0) \cong \mathcal{O}_0$.

The Deuring correspondence also preserves morphisms between these categories. To any isogeny $\varphi : E \rightarrow E'$, we can associate an ideal I_φ in $B_{p,\infty}$ such that $\mathcal{O}_L(I_\varphi) \cong \text{End}(E)$ and $\mathcal{O}_R(I_\varphi) \cong \text{End}(E')$. In this way, I_φ connects $\text{End}(E)$ to $\text{End}(E')$, just as φ connects E to E' . Moreover, $\text{Nrd}(I_\varphi) = \deg(\varphi)$. Conversely, for any left $\text{End}(E)$ -ideal I , there exists an associated isogeny φ_I such that $I_{\varphi_I} = I$ and $\varphi_{I_\varphi} = \varphi$.

E A Variant with Faster Encryption and Decryption but Slower Updates

As noted in Remark 1, we now present a variant of DINE that fully conforms to the standard UE interface, since all keys can be generated via **KeyGen**. The main modifications are highlighted in Fig. 10.

In DINE the token is made of 4 canonical HD representations $(s_{\text{HD}}^{\text{can}}(\delta_{e+1}^i))_{1 \leq i \leq 4}$ which all together represent a degree $(2^f - N)^4$ isogeny $\delta_{e+1} : E_e \rightarrow E_{e+1}$. However, when k_{e+1} is chosen beforehand, we cannot be sure to find an isogeny of exactly this degree. To address this, we instead select δ_{e+1} to have degree $(2^f - N)^k$ for some $k > 1$ using the **SpecialKLPT** algorithm. This isogeny can then be factored as the composition of k isogenies of degree $2^f - N$, denoted $\delta_{e+1,i}$ for $1 \leq i \leq k$. The actual token consists of the canonical HD representations of these component isogenies.

Concretely, the token can be computed using an algorithm we denote by **IdealToSpecialEquivalentIsogeny**, which we do not describe in full, as it can be straightforwardly extracted from **IdealPullBack**. Essentially, it outputs the successive representations s_1 computed internally during the main loop of **IdealPullBack**.

As noted in our technical overview, the security of the variant presented in this section is harder to analyze. This is because the chain of isogenies produced by **SpecialKLPT**—which forms the token—follows a highly specific distribution that is challenging to study. In contrast, in DINE, the token is uniformly random. Analyzing the output distribution of a KLPT variant was central to the security of the original SQIsign signature scheme [20]; for this reason, SQIsign’s main security assumption remains somewhat ad hoc, unlike the newer HD variants [5, 17, 1], whose security is much cleaner.

TokenGen (k_e, k_{e+1})
Parse k_e as an ideal I_e Parse k_{e+1} as an ideal I_{e+1} return $\Delta_{e+1} := (s_{\text{HD}}^{\text{can}}(\delta_{e+1,i}))_{1 \leq i \leq k} \leftarrow \text{IdealToSpecialEquivalentIsogeny}(I_e \cdot \overline{I_{e+1}})$
Upd (Δ_{e+1}, C_e)
Parse Δ_{e+1} as $(s_{\text{HD}}^{\text{can}}(\delta_{e+1,i}))_{1 \leq i \leq k}$ Parse C_e as $s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$ for $i = 1$ to k : $s_{\text{HD}}^{\text{can}}(\varphi_1) \leftarrow \text{HDPushForward}(s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\delta_{e+1,i}))$ $s_{\text{HD}}^{\text{can}}(\varphi_2) \leftarrow \text{HDPushForward}(s_{\text{HD}}^{\text{can}}(\varphi_2), s_{\text{HD}}^{\text{can}}(\delta_{e+1,i}))$ return $C_{e+1} \leftarrow s_{\text{HD}}^{\text{can}}(\varphi_1), s_{\text{HD}}^{\text{can}}(\varphi_2)$

Fig. 10. A variant with faster encryption and decryption but slower updates. All other algorithms are unchanged compared to Fig. 4.

Remark 3. This variant of DINE requires a slightly more general version of the SpecialKLPT algorithm presented in Section 5.2 that works for any input ideal, not just $\mathcal{O}_0$ -ideals. The solution follows [20] to extend the original KLPT algorithm [29]. Let I be the input ideal. The idea is to find β of the desired norm in $\mathcal{O}_0 \cap I$, which can be done via strong approximation modulo $N \text{Nrd}(K) \text{Nrd}(L)$ (instead of $N \text{Nrd}(K)$ in SpecialKLPT), where L is a connecting ideal between $\mathcal{O}_0$ and $\mathcal{O}_L(I)$. Consequently, the output norm increases roughly by a factor $\text{Nrd}(L)^3$, which we can expect to be around $p^{3/2}$ (up to replacing L by an equivalent ideal of smaller norm). For more details, see [32, Section 3.3].

Regarding the implementation results for DINE presented in Section 6.3, in the variant of DINE presented above, encryption and decryption would take roughly the same time as DINE's Upd, while updates would be slightly slower than DINE's Dec. Token generation would also be a bit slower than DINE's KeyGen, due to the slightly larger output required by the variant of SpecialKLPT.

F Proof of the detIND-UE-CPA Security of DINE

In this section, we study the detIND-UE-CPA security of DINE by giving a full proof of Theorem 3 which we recall below.

Theorem 3 (DINE is detIND-UE-CPA). *Let DINE be the UE scheme described in Fig. 4 for the special maximal order $\mathcal{O}_0$ and integer N . For any ideal cipher model adversary $\mathcal{A}$, there exist reductions $\mathcal{B}$ and $\mathcal{B}'$ such that*

$$\text{Adv}_{\text{DINE}}^{\text{detIND-UE-CPA}}(\mathcal{A}) \leq 2(n+1)^3 (\text{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{B}) + 2 \cdot \text{Adv}_{2^f - N, N}^{\text{DIPHTI}}(\mathcal{B}'))$$

We follow the proof strategy of [8, 34] and use their hybrid argument across insulated regions. In each hybrid, we can embed at one firewall of the insulated region and simulate all tokens within that insulated region to answer queries to both $\mathcal{O}.\text{Upd}$ and $\mathcal{O}.\text{Upd}\tilde{\mathcal{C}}$. In DINE, fresh ciphertexts are randomized using a nonce R but updates are deterministic. Thus, our reduction will need to provide consistent ciphertexts to the adversary, *i.e.*, the nonce value R must be consistent.

We give a reduction $\mathcal{B}$ which receives $\mathcal{O}_0, N$ and an oracle BB that implements either the distribution $\mathcal{D}_0$ or $\mathcal{D}_1$ of Definition 6, *i.e.*, it returns either tuples of the form $((\varphi_{I_1}, \varphi_{I_2}), [\varphi_K]_*(\varphi_{I_1}, \varphi_{I_2}))$ or $((\varphi_{I_1}, \varphi_{I_2}), (\varphi_{J_1}, \varphi_{J_2}))$ where K is a fixed left $\mathcal{O}_0$ -ideal, $(I_1, I_2) \leftarrow \mathcal{I}$ and J_1, J_2 are random left $\mathcal{O}$ -ideals of norm N for some random maximal order $\mathcal{O}$. $\mathcal{B}$ will use the samples of BB to perfectly simulate the detIND-UE-CPA experiment for DINE when BB implements $\mathcal{D}_0$ (and a random experiment otherwise). The idea is to embed the isogeny φ_K to a well chosen epoch key by using $\varphi_{I_1}, \varphi_{I_2}$ as randomness and $[\varphi_K]_*(\varphi_{I_1}, \varphi_{I_2})$ as ciphertext. Thus, if we know an efficient adversary $\mathcal{A}$ against the detIND-UE-CPA security of DINE, using the hybrid argument of [8], $\mathcal{B}$ can use $\mathcal{A}$ to break the pseudorandomness of the $(\mathcal{O}_0, N)$ -pushforward.

Proof. Play hybrid games. We partition the non corrupted key space as follows: $\{0, \dots, n\} \setminus \mathcal{K}^* = \cup_{(j, \text{fwl}_j, \text{fwr}_j) \in \mathcal{FW}} \{\text{fwl}_j \dots \text{fwr}_j\}$, where fwl_i and fwr_i are firewalls of the i -th insulated region. For $b \in \{0, 1\}$, define game $\mathcal{G}_i^b$ as $\mathbf{Exp}_{\text{DINE}}^{\text{detIND-UE-CPA-}b}(\mathcal{A})$ except for:

1. The game randomly picks $\text{fwl}_i, \text{fwr}_i \xleftarrow{\$} \{0, \dots, n\}$ and if they are not the i -th firewalls, it aborts and returns a random bit b' . This loss is upper-bounded by $(n+1)^2$.
2. For the challenge (made in epoch $\tilde{e}$ on input $(\bar{M}, \bar{C})$), the game returns an updated version of $\bar{C}$ if $\tilde{e} < \text{fwl}_i$ and it returns an encryption of $\bar{M}$ if $\tilde{e} > \text{fwr}_i$. Finally, if $\text{fwl}_i \leq \tilde{e} \leq \text{fwr}_i$, the game returns an encryption of $\bar{M}$ if $b = 0$ and an updated version of $\bar{C}$ if $b = 1$.
3. After $\mathcal{A}$ outputs b' , the game returns b' if $\text{twf} \neq 1$ or some additional trivial win condition triggers.

If $\text{fwl}_i, \text{fwr}_i$ are the desired values, then $\mathcal{G}_1^0$ is $\mathbf{Exp}_{\text{DINE}}^{\text{detIND-UE-CPA-0}}(\mathcal{A})$, *i.e.*, all challenges are encryptions of $\bar{M}$. Let ℓ be the total number of insulated regions (bounded by $n+1$), such that $\mathcal{G}_\ell^1$ is $\mathbf{Exp}_{\text{DINE}}^{\text{detIND-UE-CPA-1}}(\mathcal{A})$, *i.e.*, all challenges are updates of $\bar{C}$. Let E be the event that fwl_i and fwr_i are the desired values. By definition, for any $1 \leq i \leq n+1$ and $b \in \{0, 1\}$, we have $\Pr[\mathcal{G}_i^b = 1 \mid \neg E] = 1/2$. Then

$$\begin{aligned} \Pr[\mathcal{G}_\ell^1 = 1] &= \Pr[\mathcal{G}_\ell^1 = 1 \mid E] \cdot \Pr[E] + \Pr[\mathcal{G}_\ell^1 = 1 \mid \neg E] \cdot \Pr[\neg E] \\ &= \Pr[\mathbf{Exp}_{\text{DINE}}^{\text{detIND-UE-CPA-1}}(\mathcal{A}) = 1] \cdot \frac{1}{(n+1)^2} + \frac{1}{2} \cdot \left(1 - \frac{1}{(n+1)^2}\right), \text{ and} \\ \Pr[\mathcal{G}_1^0 = 1] &= \Pr[\mathbf{Exp}_{\text{DINE}}^{\text{detIND-UE-CPA-0}}(\mathcal{A}) = 1] \cdot \frac{1}{(n+1)^2} + \frac{1}{2} \cdot \left(1 - \frac{1}{(n+1)^2}\right) \end{aligned}$$

Thus, we have $|\Pr[\mathcal{G}_\ell^1 = 1] - \Pr[\mathcal{G}_1^0 = 1]| = \frac{1}{(n+1)^2} \cdot \mathbf{Adv}_{\text{DINE}}^{\text{detIND-UE-CPA}}(\mathcal{A})$.

Notice that the games $\mathcal{G}_{i-1}^1$ and $\mathcal{G}_i^0$ behave in the same way: for the challenge query and $\mathcal{O}.\text{Upd}\tilde{C}$, in an epoch in the first $i-1$ insulated regions, the reduction returns an update of $\bar{C}$, otherwise it returns an encryption of $\bar{M}$. Thus, for any $\ell \leq n+1$, $|\Pr[\mathcal{G}_\ell^1 = 1] - \Pr[\mathcal{G}_1^0 = 1]| \leq \sum_{i=1}^\ell |\Pr[\mathcal{G}_i^1 = 1] - \Pr[\mathcal{G}_i^0 = 1]|$. In the following, we prove that for any $1 \leq i \leq \ell$, $|\Pr[\mathcal{G}_i^1 = 1] - \Pr[\mathcal{G}_i^0 = 1]| \leq 2\mathbf{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{B})$ for a reduction $\mathcal{B}$.

In hybrid i. Let $\mathcal{A}_i$ be an adversary trying to distinguish $\mathcal{G}_i^0$ from $\mathcal{G}_i^1$. For all queries concerning epochs outside of the i -th insulated region, the responses of both games are the same. Thus, we assume that $\mathcal{A}_i$ asks for at least one challenge ciphertext in an epoch within the i -th insulated region. This is where we will embed the pseudorandom pushforward samples in our reduction.

We construct a reduction $\mathcal{B}$, presented in Fig. 11, that is trying to break the pseudorandomness of the $(\mathcal{O}_0, N)$ -pushforward (Definition 6) and will simulate the responses of queries made by adversary $\mathcal{A}_i$.

Recall that $\mathcal{A}_i$ is an adversary attempting to distinguish $\mathcal{G}_i^0$ from $\mathcal{G}_i^1$. $\mathcal{B}$ will use $\mathcal{A}$ to break the pseudorandomness of the $(\mathcal{O}_0, N)$ -pushforward. In $\mathbf{Exp}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{B})$,

Reduction $\mathcal{B}$ playing $\text{Exp}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}$	$\mathcal{O}.\text{Enc}(M)$	$\mathcal{O}.\text{Chall}(\bar{M}, \bar{C})$
receive pp, and oracle BB do Setup $\bar{M}, \bar{C} \leftarrow \mathcal{A}^{\text{ors}}(\text{pp})$ phase $\leftarrow 1$ $\bar{C}_{\bar{e}} \leftarrow \mathcal{O}.\text{Chall}(\bar{M}, \bar{C})$ $b' \leftarrow \mathcal{A}^{\text{ors}, \mathcal{O}.\text{Upd}\bar{C}}(\bar{C}_{\bar{e}})$ if $\mathcal{C}^* \cap \mathcal{K}^* \neq \emptyset$ or $\mathcal{I}^* \cap \mathcal{C}^* \neq \emptyset$ twf $\leftarrow 1$ if ABORT occurred or twf = 1 $b' \xleftarrow{\$} \{0, 1\}$ return b' if $(i, \text{fwl}_i, \text{fwr}_i) \notin \mathcal{FW}$ $b' \xleftarrow{\$} \{0, 1\}$ return b' if $b' = b$ then return 0 else return 1 Setup $b \xleftarrow{\$} \{0, 1\}$ $k_0 \leftarrow \text{DINE.KeyGen}(\text{pp})$ $\Delta_0 \leftarrow \perp$ $e, c, \text{phase}, \text{twf} \leftarrow 0$ $\mathcal{L}, \tilde{\mathcal{L}}, \mathcal{C}, \mathcal{K}, \mathcal{T} \leftarrow \emptyset$ $\text{fwl}_i, \text{fwr}_i \xleftarrow{\$} [0, n]$ $k_{\text{fwr}_i+1} \leftarrow \text{DINE.KeyGen}(\text{pp})$ for $j \in [0, \text{fwl}_i - 1] \cup [\text{fwr}_i + 2, n]$: $k_j, \Delta_j \leftarrow \text{DINE.UpdateKey}(k_{j-1})^{\bowtie}$ for $j \in [\text{fwl}_i + 1, \text{fwr}_i]$: $\Delta_j \xleftarrow{\$} \text{Iso}_{2^f - N}(\text{codomain}(k_{j-1}))$ $\mathcal{O}.\text{Next}$ $e \leftarrow e + 1$	$c \leftarrow c + 1$ $(\text{inf}_1, \text{inf}_2) \leftarrow \text{BB}$ $\pi(R M) \leftarrow \text{HETC}(\text{inf}_1)$ if $e \in [0, \text{fwl}_i - 1] \cup [\text{fwr}_i + 1, n]$ $C_e \leftarrow [k_e]_* \text{inf}_1$ else $C_{\text{fwl}_i} \leftarrow \text{inf}_2$ for $j \in [\text{fwl}_i + 1, e]$ $C_j \leftarrow [\Delta_j]_* C_{j-1}$ $\text{inf} \leftarrow (\text{inf}_1, \text{inf}_2)$ $\mathcal{L} \leftarrow \mathcal{L} \cup \{(c, C_e, e; \text{inf})\}$ return C_e $\mathcal{O}.\text{Upd}(C_{e-1})$ if $(c, C_{e-1}, e - 1; \text{inf}) \notin \mathcal{L}$ return $\perp$ if $e \in [1, \text{fwl}_i - 1] \cup [\text{fwr}_i + 1, n]$ $(\text{inf}_1, \text{inf}_2) \leftarrow \text{inf}$ $C_e \leftarrow [k_e]_* \text{inf}_1$ else $(\text{inf}_1, \text{inf}_2) \leftarrow \text{inf}$ $C_{\text{fwl}_i} \leftarrow \text{inf}_2$ for $j \in [\text{fwl}_i + 1, e]$ $C_j \leftarrow [\Delta_j]_* C_{j-1}$ $\mathcal{L} \leftarrow \mathcal{L} \cup \{(c, C_e, e; \text{inf})\}$ return C_e $\mathcal{O}.\text{Corr}(\text{inp}, \hat{e})$ if $\text{inp} = \text{key}$ $\mathcal{K} \leftarrow \mathcal{K} \cup \{\hat{e}\}$ return $k_{\hat{e}}$ if $\text{inp} = \text{token}$ $\mathcal{T} \leftarrow \mathcal{T} \cup \{\hat{e}\}$ return $\Delta_{\hat{e}}$	if $(c, \bar{C}, \bar{e} - 1; \text{inf}) \notin \mathcal{L}$ return ABORT if $b = 0$ $(s_1, s_2) \leftarrow \text{BB}$ $\pi(R \bar{M}) \leftarrow \text{HETC}(s_1)$ $\tilde{C}_{\text{fwl}_i} \leftarrow s_2$ else $(\text{inf}_1, \text{inf}_2) \leftarrow \text{inf}$ do $(z, t) \xleftarrow{\$} [-N/\sqrt{2p}, N/\sqrt{2p}]^2$ while $\text{HETC}(z, t) \neq \perp$ $\pi(R \bar{M}) \leftarrow \text{HETC}(z, t)$ $\tilde{C}_{\text{fwl}_i} \leftarrow \text{inf}_2$ for $j \in [0, \text{fwl}_i - 1]$: // left $\tilde{C}_j \leftarrow [\bigcirc_{k=j}^1 \Delta_k]_* [\bigcirc_{k=\bar{e}-1}^1 \Delta_k]_* \tilde{C}$ for $j \in [\text{fwl}_i + 1, \text{fwr}_i]$: // embed $\tilde{C}_j \leftarrow [\Delta_j]_* \tilde{C}_{j-1}$ for $j \in [\text{fwr}_i + 1, n]$: // right $\tilde{C}_j \leftarrow [k_j]_* \pi(R \bar{M})$ $\tilde{\mathcal{L}} \leftarrow \bigcup_{j=0}^n \{(\tilde{C}_j, j)\}$ return $\tilde{C}_e$ $\mathcal{O}.\text{Upd}\tilde{C}$ $\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$ find $(\tilde{C}_e, e) \in \tilde{\mathcal{L}}$ return $\tilde{C}_e$

Fig. 11. Our reduction $\mathcal{B}$ for proof of Th.3 in hybrid i . inf encodes fixed programming information: two isogenies pairs $(\text{inf}_1, \text{inf}_2)$ sampled by BB, where inf_1 is the encryption randomness and inf_2 is the ciphertext value in epoch fwl_i . **ors** denotes the set $\{\mathcal{O}.\text{Enc}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{Upd}, \mathcal{O}.\text{Corr}\}$. $\bowtie$ indicates that Δ_0 and Δ_{fwr_i+1} are skipped in the computation. HETC outputs the last two values returned by HalfEndoToCoord, and $\text{Iso}_{2^f - N}(E)$ is the uniform distribution over $(2^f - N)$ -isogenies with domain E .

when BB returns samples of the form $(\varphi_{I_1}, \varphi_{I_2}, [\varphi_K]_* \varphi_{I_1}, [\varphi_K]_* \varphi_{I_2})$ for a random left $\mathcal{O}_0$ -ideal K and $(I_1, I_2) \leftarrow \mathcal{I}$, $\mathcal{B}$ will perfectly simulate the environment of $\mathcal{A}_i$ in $\mathcal{G}_i^b$. When BB returns pairs of the form $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2})$ for $(I_1, I_2) \leftarrow \mathcal{I}$ and J_1, J_2 random left $\mathcal{O}$ -ideals of norm N for some random maximal order $\mathcal{O}$, $\mathcal{B}$ will give random inputs to $\mathcal{A}_i$ such that $\mathcal{A}_i$ distinguishes $\mathcal{G}_i^0$ from $\mathcal{G}_i^1$ with advantage 0. We explain how our reduction $\mathcal{B}$ does this without knowing which BB oracle was provided to it.

The reduction $\mathcal{B}$ receives the oracle BB, takes $b \xleftarrow{\$} \{0, 1\}$ and simulates $\mathcal{G}_i^b$. Whenever the reduction needs to provide an output of $\pi(\cdot)$ to $\mathcal{A}_i$, it chooses some integers $z, t \in [-N/\sqrt{2p}, N/\sqrt{2p}]$ and programs $\pi(\cdot) = (z, t)$. In this setting, computing π^{-1} is simply a lookup to this mapping of the ideal cipher π . We now explain our simulation. Initially,

1. $\mathcal{B}$ guesses the values of fwl_i and fwr_i .
2. $\mathcal{B}$ generates all keys and tokens except for $k_{\text{fwl}_i}, \dots, k_{\text{fwr}_i}, \Delta_{\text{fwl}_i}, \Delta_{\text{fwr}_i+1}$. If $\mathcal{A}_i$ corrupts these keys and tokens, this means that the firewall guess is wrong and the reduction aborts the game using the Check algorithm of Appendix B.

$\mathcal{B}$ will operate so as to embed the value φ_K used by BB, when it implements $\mathcal{D}_0$, to the key k_{fwl_i} . If BB implements $\mathcal{D}_1$ instead, all the ciphertexts inside insulated region i will be random (no key or token could possibly explain these ciphertexts). To simulate a non-challenge ciphertext that is:

- An $\mathcal{O}.\text{Enc}$ query in epoch $e \in \{0, \dots, \text{fwl}_i - 1\} \cup \{\text{fwr}_i + 1, \dots, n\}$: $\mathcal{B}$ queries BB to get a tuple $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2})$. $\mathcal{B}$ uses $\varphi_{I_1}, \varphi_{I_2}$ as a random value by computing the quaternion $\theta := x + yi + zj + tj \in B_{p,\infty}$ through the HalfEndoToCoord algorithm called on $\varphi_{I_1}, \varphi_{I_2}$ and programming $\pi(\cdot) \leftarrow (z, t)$ (so the randomness will be consistent with calls that $\mathcal{A}_i$ makes to $\mathcal{O}.\text{Upd}$), computes the ciphertext $C_e := ([k_e]_* \varphi_{I_1}, [k_e]_* \varphi_{I_2})$ (the value of k_e is known to $\mathcal{B}$ in these epochs) and stores $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2})$ in its memory for later use. To respond to $\mathcal{O}.\text{Upd}$ queries in these epochs, $\mathcal{B}$ computes $C_e := ([k_e]_* \varphi_{I_1}, [k_e]_* \varphi_{I_2})$ using the randomness $\varphi_{I_1}, \varphi_{I_2}$ generated during the first encryption of the input ciphertext.
- An $\mathcal{O}.\text{Enc}$ query in epoch $e \in \{\text{fwl}_i, \dots, \text{fwr}_i\}$: $\mathcal{B}$ queries BB to get a tuple $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2})$ and programs $\pi(\cdot)$ using $\varphi_{I_1}, \varphi_{I_2}$ as above. It sets $C_{\text{fwl}_i} = (\varphi_{J_1}, \varphi_{J_2})$ (so that all ciphertexts will be encrypted under the key φ_K in epoch fwl_i if BB returns tuples of the form $(\varphi_{I_1}, \varphi_{I_2}, [\varphi_K]_* \varphi_{I_1}, [\varphi_K]_* \varphi_{I_2})$) and updates C_{fwl_i} to the right epoch e using its simulated tokens (remember that $\mathcal{B}$ does not know the keys inside the i -th insulated region). To respond to $\mathcal{O}.\text{Upd}$ queries in these epochs, $\mathcal{B}$ uses the values $\varphi_{J_1}, \varphi_{J_2}$ (if $\varphi_{J_i} = [\varphi_K]_* \varphi_{I_i}$ then the randomness will still be consistent) generated during the first encryption of the input ciphertext as ciphertext value in epoch fwl_i and updates $\varphi_{J_1}, \varphi_{J_2}$ to the right epoch e using its simulated tokens.

During the challenge call, the adversary will provide a ciphertext $\bar{C}$ which was created during the c -th call to $\mathcal{O}.\text{Enc}$. The adversary cannot ask for an update of the c -th encryption in an epoch $e \geq \text{fwl}_i$, as this would trigger the trivial win condition $[\text{fwl}_i, \text{fwr}_i] \subseteq \mathcal{I}^* \cap \mathcal{C}^* \neq \emptyset$.

To simulate challenge-equal ciphertext in an epoch that is:

- To the left of the i -th insulated region: $\mathcal{B}$ simulates $\text{DINE.Upd}(\bar{C})$ using tokens that it created itself.
- Within the i -th insulated region: $\mathcal{B}$ simulates $\text{DINE.Upd}(\bar{C})$ if $b = 1$, and simulates $\text{DINE.Enc}(\bar{M})$ if $b = 0$. More precisely, if BB returns tuples of the form $(\varphi_{I_1}, \varphi_{I_2}, [\varphi_K]_* \varphi_{I_1}, [\varphi_K]_* \varphi_{I_2})$, $\mathcal{B}$ embeds φ_K to k_{fwl_i} . If $b = 0$, the reduction samples $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2}) \leftarrow \text{BB}$, programs $\pi(R||\bar{M})$ using the **HalfEndoToCoord** algorithm called on $(\varphi_{I_1}, \varphi_{I_2})$ as before and sets $\bar{C}_{\text{fwl}_i} = (\varphi_{J_1}, \varphi_{J_2})$ (we want $k_{\text{fwl}_i} = \varphi_K$) since $\bar{C}_{\text{fwl}_i} = \text{DINE.Enc}(\bar{M}) = ([k_{\text{fwl}_i}]_* \varphi_{I_1}, [k_{\text{fwl}_i}]_* \varphi_{I_2})$ by definition of $\pi(R||\bar{M})$.
If $b = 1$, assume that $\bar{C}$ is an update of $\bar{C}_{e_c}$, the output of the c -th $\mathcal{O}.\text{Enc}$ query. $\mathcal{B}$ sampled $(\varphi_{I_1}, \varphi_{I_2}, \varphi_{J_1}, \varphi_{J_2})$ using BB and used $\varphi_{I_1}, \varphi_{I_2}$ as randomness to create $\bar{C}_{e_c}$ and to update it in epochs $e < \text{fwl}_i$. The reduction gives value $\varphi_{J_1}, \varphi_{J_2}$ to $\bar{C}_{\text{fwl}_i}$ since $\bar{C}_{\text{fwl}_i} = \text{DINE.Upd}(\bar{C}) = ([\Delta_{\text{fwl}_i}]_* ([k_{\text{fwl}_i-1}]_* \varphi_{I_1}), [\Delta_{\text{fwl}_i}]_* ([k_{\text{fwl}_i-1}]_* \varphi_{I_2})) = ([k_{\text{fwl}_i}]_* \varphi_{I_1}, [k_{\text{fwl}_i}]_* \varphi_{I_2})$ by the correctness of updates of DINE (see Proposition 1). Furthermore, the reduction uses tokens $\Delta_{\text{fwl}_i+1}, \dots, \Delta_{\text{fwl}_i}$ to update $\bar{C}_{\text{fwl}_i}$ to simulate all challenge ciphertexts in epochs within the insulated region.
- To the right of the i -th insulated region: $\mathcal{B}$ simulates $\text{DINE.Enc}(\bar{M})$ using the keys that it created itself.

Eventually, $\mathcal{B}$ receives the output bit b' from $\mathcal{A}_i$. If $b' = b$, then $\mathcal{B}$ guesses that BB implements $\mathcal{D}_0$; otherwise, $\mathcal{B}$ guesses that BB implements $\mathcal{D}_1$. If $\mathcal{B}$ received an oracle BB implementing $\mathcal{D}_1$, then $\mathcal{B}$ wins with probability $1/2$, since the ciphertexts are completely random. If $\mathcal{B}$ perfectly simulated the view of $\mathcal{A}_i$ in $\mathcal{G}_i^b$ when BB implements $\mathcal{D}_0$, a standard advantage computation would yield $\text{Adv}_{\text{DINE}}^{\text{detIND-UE-CPA}}(\mathcal{A}) \leq 2(n+1)^3 \text{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{B})$.

However, perfect simulation does not quite hold. The issue is that the key φ_K used by $\mathcal{D}_0$ and embedded in epoch fwl_i is generated using **KeyGen**, whereas in DINE keys are derived using **UpdateKey**. Hence, we have to justify that this change does not affect the success probability of $\mathcal{A}_i$ in $\mathcal{G}_i^b$ too much. In particular, generating the key of epoch fwl_i using **KeyGen** instead of **UpdateKey** means that it is possible that no suitable token isogeny exists between E_{fwl_i-1} and E_{fwl_i} . To analyze this gap, we introduce the following game hop: game $\mathbf{G}_0$ is $\mathcal{G}_i^b$ and game $\mathbf{G}_1$ is identical to $\mathbf{G}_0$, except the key in epoch fwl_i is generated using **KeyGen** instead of **UpdateKey**.

If there exists an admissible token for the key generated by **KeyGen**, then the two games are indistinguishable (we do not need to know this token since it is never revealed to the adversary because fwl_i is the left firewall of an insulated region). We thus consider the case where no such admissible token exists.

Let $\mathcal{A}'$ be an adversary distinguishing $\mathbf{G}_0$ from $\mathbf{G}_1$. We construct a reduction $\mathcal{B}'$ that uses $\mathcal{A}'$ to solve the **DIPHTI** problem of Definition 8. $\mathcal{B}'$ sends the curve E_{fwl_i-1} to the challenger and receives back a curve E' together with oracle access to a bijection σ_b between the sets of N -isogenies with domain E_{fwl_i-1} and E' , where:

- if $b = 0$, then E' is $(2^f - N)$ -isogenous to E_{fwl_i-1} via ψ , and σ_0 returns pushforwards through ψ ;
- if $b = 1$, then E' is uniform and σ_1 is a random bijection.

$\mathcal{B}'$ sets $E_{\text{fwl}_i} := E'$ and embeds ψ as the update token between E_{fwl_i-1} and E_{fwl_i} . To update a ciphertext $C = (\varphi_1, \varphi_2)$ from epoch $\text{fwl}_i - 1$ to fwl_i , $\mathcal{B}'$ computes $C' := (\sigma_b(\varphi_1), \sigma_b(\varphi_2))$.

- When $b = 0$, this exactly matches the update procedure of DINE and, by correctness, $\mathcal{B}'$ perfectly simulates $\mathbf{G}_0$.
- When $b = 1$, the ciphertexts are totally random, and since we assume no admissible token exists, the view of the adversary is identical to that of $\mathbf{G}_1$.

Thus,

$$|\Pr[\mathcal{A}'^{\mathbf{G}_0} = 1] - \Pr[\mathcal{A}'^{\mathbf{G}_1} = 1]| \leq \mathbf{Adv}_{2^f-N, N}^{\text{DIPHTI}}(\mathcal{B}').$$

We apply the exact same argument to the epoch key $\text{fwr}_i + 1$. Indeed, when exiting the i -th insulated region in the hybrid argument, the reduction $\mathcal{B}$ does not know the key of epoch fwr_i , and therefore must generate the key of epoch $\text{fwr}_i + 1$ using KeyGen. Putting everything together, we get

$$\mathbf{Adv}_{\text{DINE}}^{\text{detIND-UE-CPA}}(\mathcal{A}) \leq 2(n+1)^3 (\mathbf{Adv}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{PP}}(\mathcal{B}) + 2 \cdot \mathbf{Adv}_{2^f-N, N}^{\text{DIPHTI}}(\mathcal{B}'))$$

G Reduction for the Proof of Theorem 4

The reduction for the proof of Theorem 4 is shown in Fig. 12.

H Proof of Proposition 3

Proof. The running time follows from the polynomial complexity of SpecialKLPT and the complexity of AnyIdealTolsogeny. The matrix M can be computed in time $O(\text{poly}(\log(p)))$ via discrete logarithms, as the points used in the HD representation have orders that are powers of two.

For correctness, we first show that the pullback of ψ through φ_J coincides with the pullback through φ_I . This follows from the fact that I and J are equivalent as $\mathbb{Z} + N\mathcal{O}_0$ -ideals, as established in the proof of Proposition 1. This equivalence can be assumed due to the correctness of SpecialKLPT.

Now, factor $\varphi_J = \varphi_e \circ \dots \circ \varphi_1$, where each φ_i has degree $2^f - N$. The pullback of ψ through this chain of e isogenies equals the iterated pushforward of ψ through $\hat{\varphi}_1, \dots, \hat{\varphi}_e$. Let ψ_i denote the iterated pushforwards, with $\psi_0 = \psi$ and $\psi_1 = [\hat{\varphi}_e]_* \psi_0$, and so on. We must show that, before executing HDPushForward at step i of the loop, s_1 correctly represents $\hat{\varphi}_e \circ \dots \circ \hat{\varphi}_{e-i+1}$, and s correctly represents ψ_{i-1} , with both having the same domain basis.

Before the loop, by the correctness of the input representation, we have $P', Q' = \psi(P, Q)$. After applying the matrix M , linearity of the isogeny action ensures that $R', S' = \psi(R, S)$, so at this stage s contains a valid representation

Reduction $\mathcal{B}$ playing $\text{Exp}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{UP}}$	$\mathcal{O}.\text{Enc}(M)$	$\mathcal{O}.\text{Corr}(\text{inp}, \hat{e})$
receive pp, BB do Setup $\mathcal{A}^{\text{ors}}(\text{pp})$ if ABORT occurred or twf = 1 win $\leftarrow$ 0 else return win Setup $k_0 \leftarrow \text{DINE.KeyGen}(\text{pp})$ $\Delta_0 \leftarrow \perp$ $e, c, \text{phase}, \text{win}, \text{twf} \leftarrow 0$ $\mathcal{L}^*, \mathcal{C}, \mathcal{K}, \mathcal{T} \leftarrow \emptyset$ $\hat{\text{fwl}} \xleftarrow{\$} [0, n]$ for $j \in [1, \hat{\text{fwl}} - 1]$: $k_j, \Delta_j \leftarrow \text{DINE.UpdateKey}(k_{j-1})$ for $j \in [\hat{\text{fwl}} + 1, n]$: $\Delta_j \xleftarrow{\$} \text{Iso}_{2^f - N}(\text{codomain}(k_{j-1}))$ $\mathcal{O}.\text{Next}$ $e \leftarrow e + 1$	$c \leftarrow c + 1$ $(\text{inf}_1, \text{inf}_2) \leftarrow \text{BB}$ if $e \in [0, \hat{\text{fwl}} - 1]$ $\pi(R \ M) \leftarrow \text{HETC}(\text{inf}_1)$ $C_e \leftarrow [k_e]_* \text{inf}_1$ else $C_{\hat{\text{fwl}}} \leftarrow \text{inf}_2$ for $j \in [\hat{\text{fwl}} + 1, e]$: $C_j \leftarrow [\Delta_j]_* C_{j-1}$ $\text{inf} \leftarrow (\text{inf}_1, \text{inf}_2)$ $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{(c, C_e, e; \text{inf})\}$ return C_e $\mathcal{O}.\text{Upd}(C_{e-1})$ if $(c, C_{e-1}, e - 1; \text{inf}) \notin \mathcal{L}^*$ return $\perp$ if $e \in [1, \hat{\text{fwl}} - 1]$ $(\text{inf}_1, \text{inf}_2) \leftarrow \text{inf}$ $C_e \leftarrow [k_e]_* \text{inf}_1$ else $(\text{inf}_1, \text{inf}_2) \leftarrow \text{inf}$ $C_{\hat{\text{fwl}}} \leftarrow \text{inf}_2$ for $j \in [\hat{\text{fwl}} + 1, e]$: $C_j \leftarrow [\Delta_j]_* C_{j-1}$ $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{(c, C_e, e; \text{inf})\}$ return C_e	do Check(inp, $\hat{e}$; e ; $\hat{\text{fwl}}, n$) if inp = key $\mathcal{K} \leftarrow \mathcal{K} \cup \{\hat{e}\}$ return $k_{\hat{e}}$ if inp = token $\mathcal{T} \leftarrow \mathcal{T} \cup \{\hat{e}\}$ for $i \in \mathcal{T}^*$: for $(j, C_{i-1}, i - 1; \text{inf}) \in \mathcal{L}^*$: $C_i \leftarrow \mathcal{O}.\text{Upd}(C_{i-1})$ $\mathcal{L}^* \leftarrow \mathcal{L}^* \cup \{(j, C_i, i; \text{inf})\}$ return $\Delta_{\hat{e}}$ $\mathcal{O}.\text{Try}(\tilde{C})$ if phase = 1 return $\perp$ phase $\leftarrow$ 1 if $\hat{e} \in \mathcal{K}^*$ or $\tilde{C} \in \mathcal{L}^*$ twf $\leftarrow$ 1 if $\hat{e} \notin [\hat{\text{fwl}}, \hat{\text{fwl}}] // \text{insulated region}$ twf $\leftarrow$ 1 $\psi \leftarrow [\mathcal{O}_{e=\hat{e}}^{\hat{\text{fwl}}+1} \Delta_e]^* \tilde{C}$ output ψ to $\text{Exp}_{\mathcal{O}_0, N, \mathcal{I}}^{\text{UP}}(\mathcal{B})$ get b win $\leftarrow b$ abort

Fig. 12. Our reduction $\mathcal{B}$ for the proof of Th.4. ors denotes the set $\{\mathcal{O}.\text{Enc}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{Upd}, \mathcal{O}.\text{Corr}, \mathcal{O}.\text{Try}\}$. HETC outputs the last two values of HalfEndoToCoord, and $\text{Iso}_{2^f - N}(E)$ is the uniform distribution over $(2^f - N)$ -isogenies with domain E .

of ψ_0 , with $R, S = \varphi_J(P_0, Q_0)$. In the first iteration of the loop, the Deuring correspondence implies that the ideal K_2 corresponds to φ_e and K_1 corresponds to $\varphi_{K_1} := \varphi_{e-1} \circ \dots \circ \varphi_1$. Thus, we have $\hat{\varphi}_e(R, S) = [2^f - N]\varphi_{K_1}(P_0, Q_0)$. This confirms that, during the first iteration of the loop, the representations stored in s and s_1 satisfy the desired property.

After executing **HDPushForward**, by the correctness of that algorithm, s contains a representation of ψ_1 , with the codomain basis given by $[2^f - N]\varphi_{e-1} \circ \dots \circ \varphi_1(P_0, Q_0)$. The argument for the correctness of the inputs to **HDPushForward** at each subsequent iteration of the loop follows analogously. $\square$